

Physical Memory (Physical Page Frame) Management in Linux Kernel

Adrian Huang | July, 2022

- * Based on kernel 5.11 (x86_64) – QEMU
- * 1-socket CPUs (8 cores/socket)
- * 16GB memory
- * Kernel parameter: nokaslr norandmaps
- * Userspace: ASLR is disabled
- * Legacy BIOS

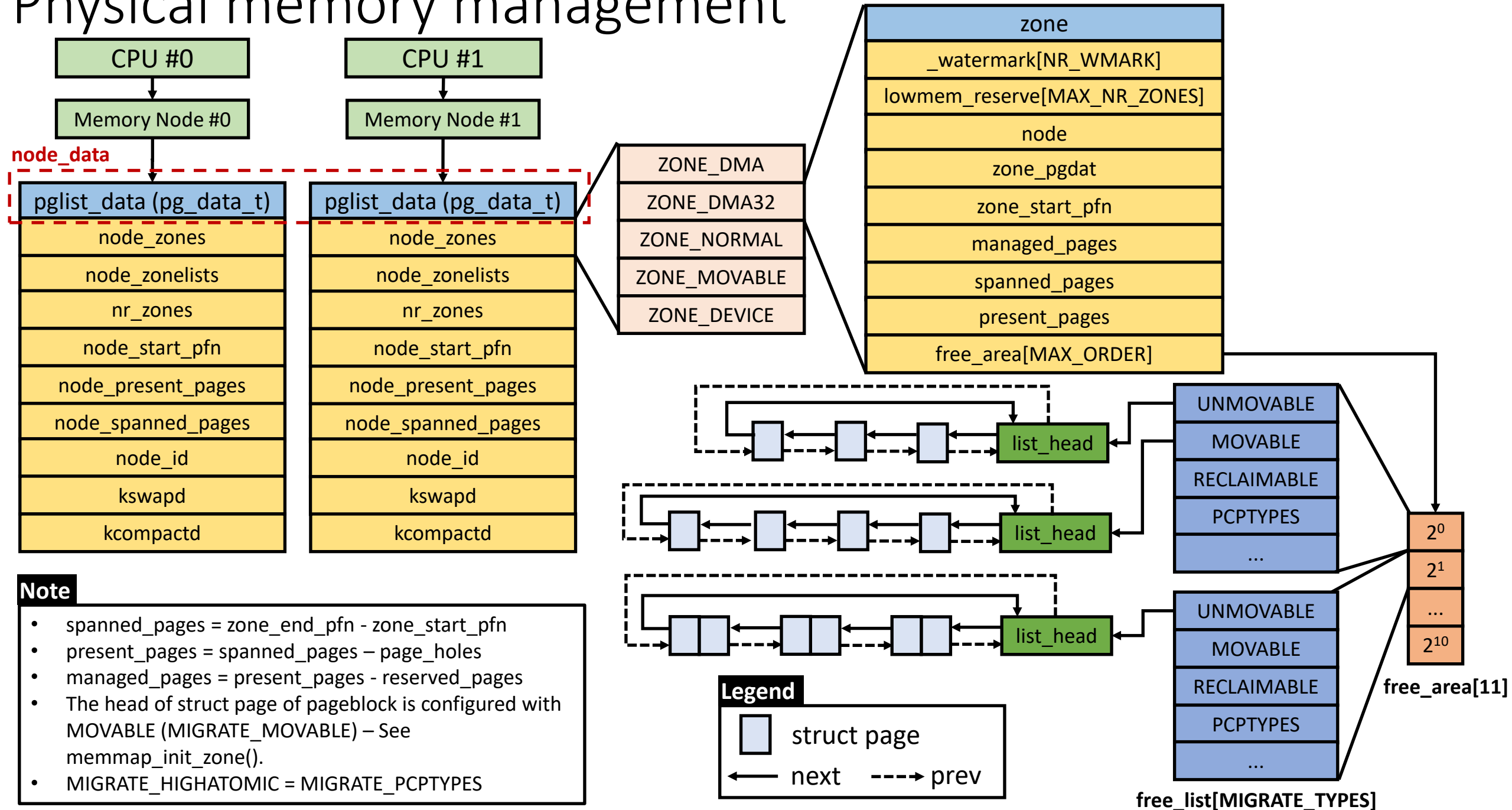
Agenda

- Physical memory (physical page frame) management overview
 - ✓ Data structures about node, zone, zonelist, migrate type and per-cpu page frame cache (per_cpu_pageset struct: PCP)
 - ✓ Placement of physical page frames right after system initialization
- Physical memory defragmentation (anti-fragmentation): Approaches
- Physical memory defragmentation (anti-fragmentation): Example
 - ✓ How the migration works
 - ✓ How the buddy system works
- pageblock
- dmesg output: total pages
- Page frame allocator
 - ✓ Watermark
 - ✓ Call Path

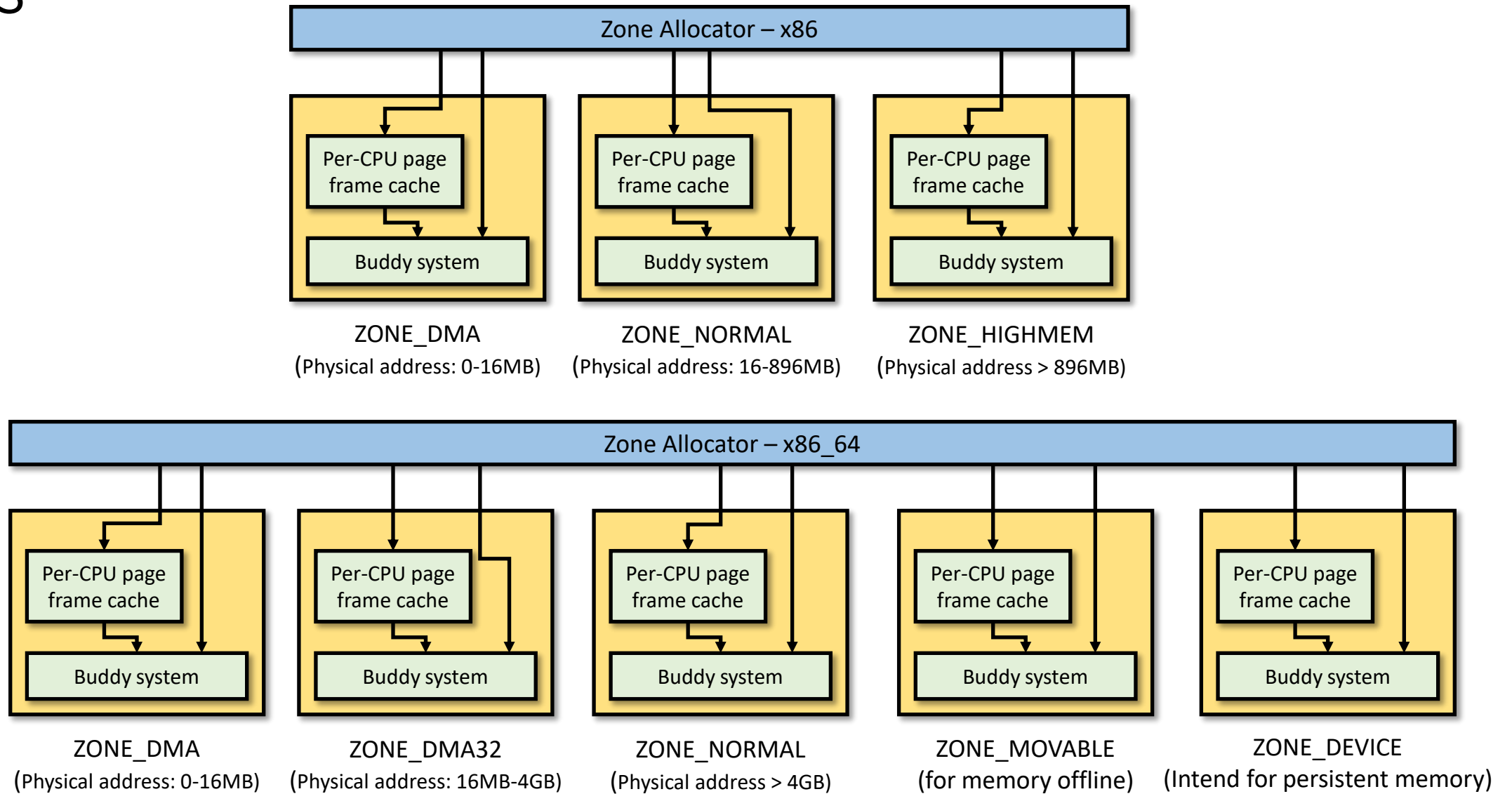
Agenda

- Does not cover in this talk
 - ✓ Memory compaction, page reclaiming and OOM killer
 - Will be discussion in the future

Physical memory management

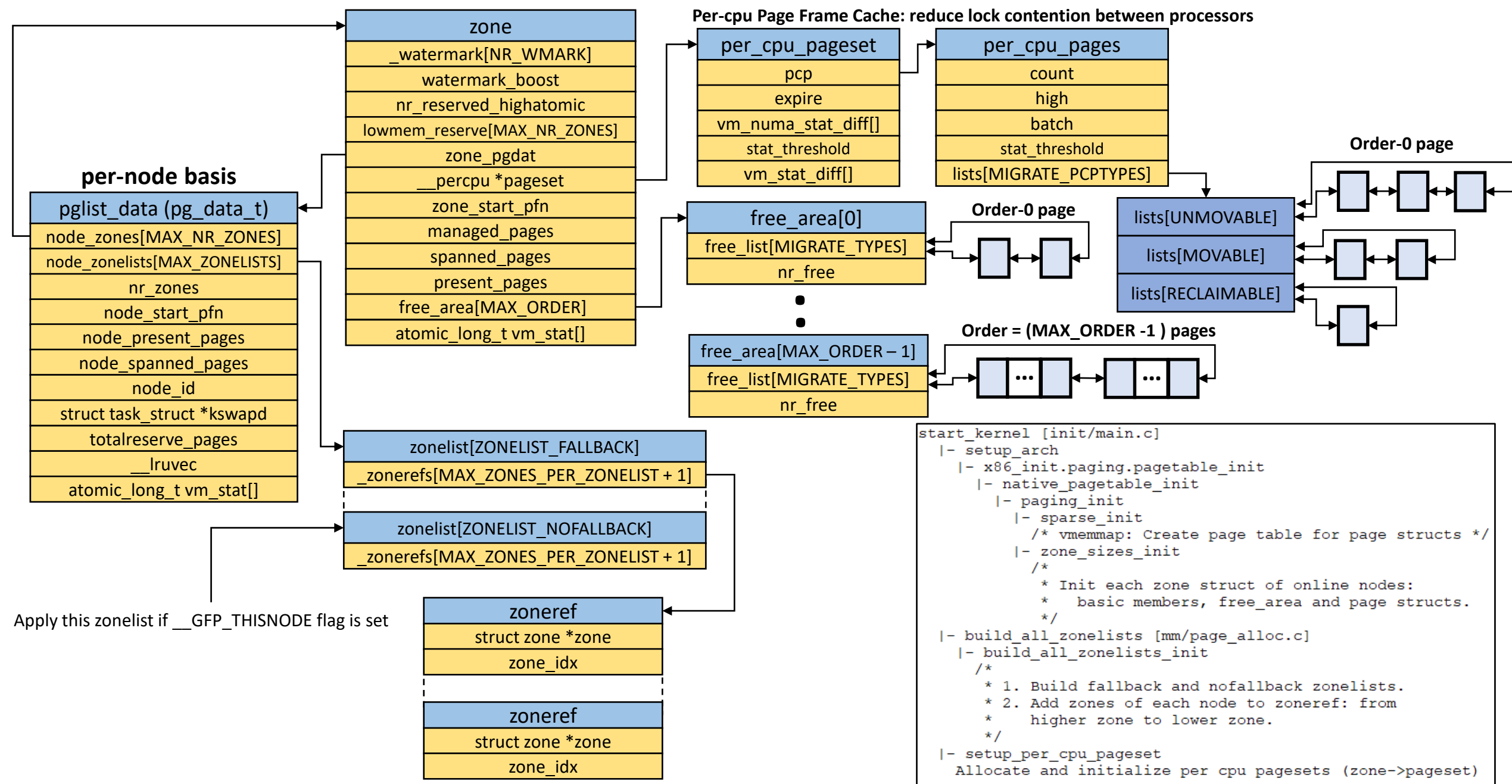


Zones

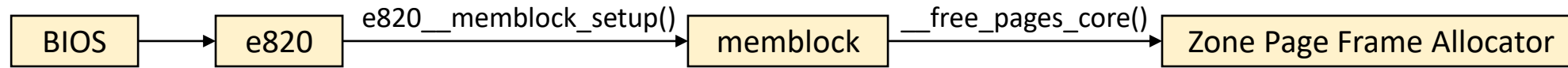


- ZONE_MOVABLE is similar to ZONE_NORMAL, except that it contains movable pages with few exceptional cases: `include/linux/mmzone.h`

Physical memory management – Zone detail



How to know available memory pages in a system?

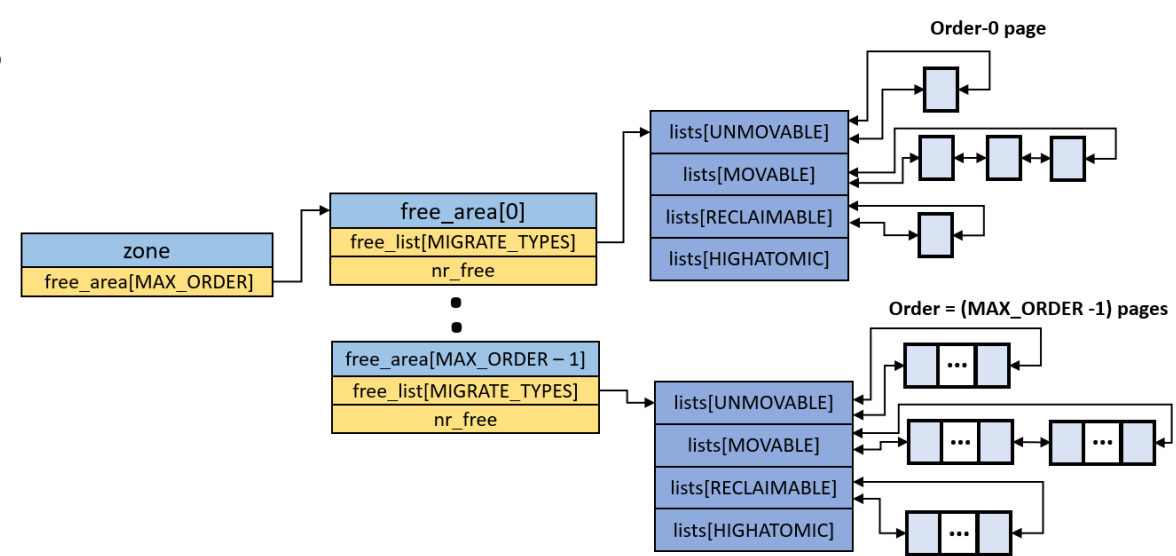


[Call Path] memblock frees available memory space to zone page frame allocator

```
start_kernel [init/main.c]
|- mm_init
  |- mem_init
    |- memblock_free_all
      |- free_low_memory_core_early
        for_each_reserved_mem_range(i, &start, &end)
          reserve_bootmem_region(start, end);
        for_each_free_mem_range(...)
          count += __free_memory_core(start, end);
```

```
__free_memory_core [mm/memblock.c]
|- __free_pages_memory
  |- memblock_free_pages [mm/page_alloc.c]
    |- __free_pages_core
      Clear flag 'PG_reserved' of each page struct
      Add number of pages to zone.managed_pages
```

Placement of physical page frames right after system initialization



```
/ # cat /proc/pagetypeinfo
```

```
Page block order: 9
```

```
Pages per block: 512
```

Free pages count per migrate type at order				0	1	2	3	4	5	6	7	8	9	10
Node 0, zone DMA32, type Unmovable				0	1	0	1	0	0	1	0	1	0	0
Node 0, zone DMA32, type Movable				2	3	7	6	4	4	4	4	3	3	745
Node 0, zone DMA32, type Reclaimable				0	0	0	0	0	0	0	0	0	0	0
Node 0, zone DMA32, type HighAtomic				0	0	0	0	0	0	0	0	0	0	0
Node 0, zone Normal, type Unmovable				1	2	1	0	0	1	0	0	0	1	0
Node 0, zone Normal, type Movable				1	1	0	1	1	0	3	0	1	1	1235
Node 0, zone Normal, type Reclaimable				1	1	0	1	1	0	1	1	1	0	0
Node 0, zone Normal, type HighAtomic				0	0	0	0	0	0	0	0	0	0	0

Number of blocks type		Unmovable	Movable	Reclaimable	HighAtomic
Node 0, zone DMA32		1	1535	0	0
Node 0, zone Normal		16	2544	2	0

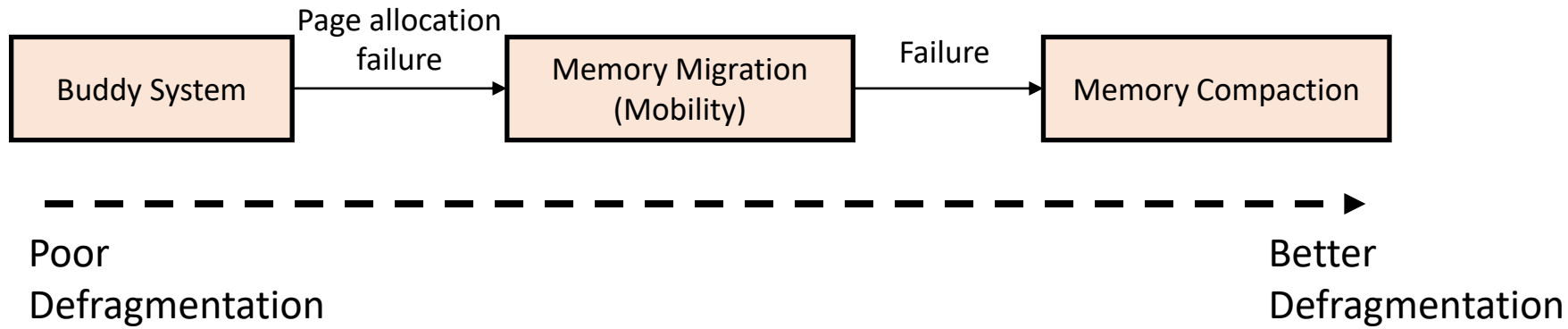
① Most physical page frames are stored in MIGRATE_MOVABLE

② [Right after system boots] Most physical page frames are stored in MAX_ORDER (order=10)

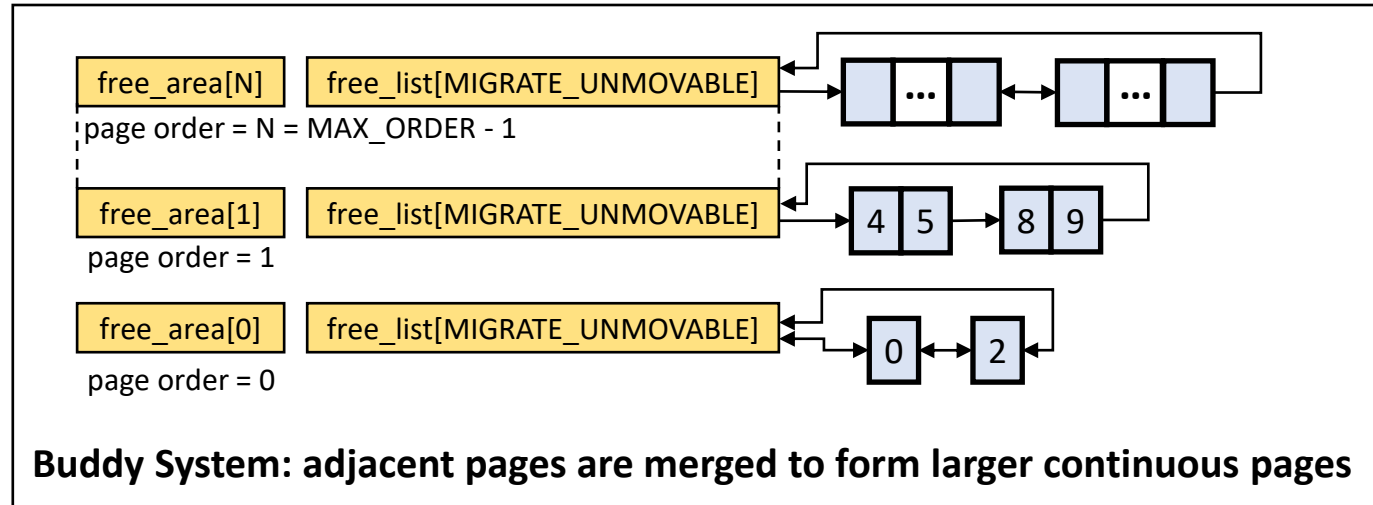
Physical memory defragmentation (anti-fragmentation): Approaches

1. Buddy system
2. Memory migration (mobility)
3. Memory compaction

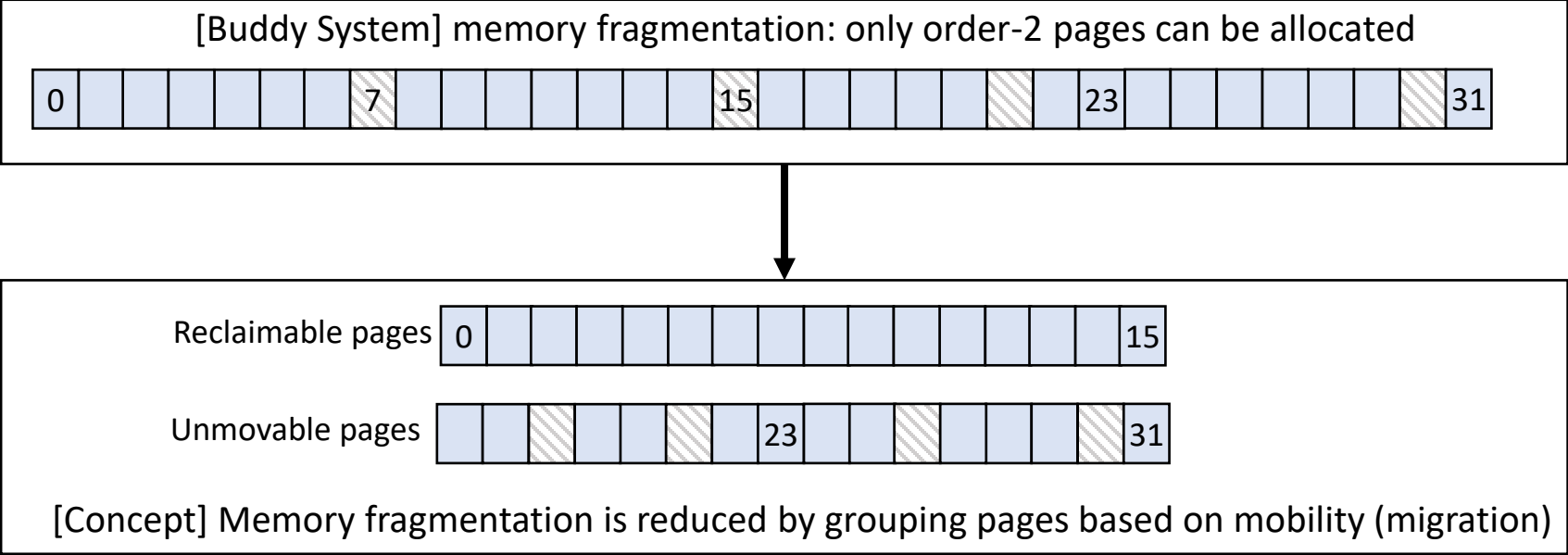
Physical memory defragmentation (Anti-fragmentation): Approaches



Physical memory defragmentation (Anti-fragmentation): Buddy System



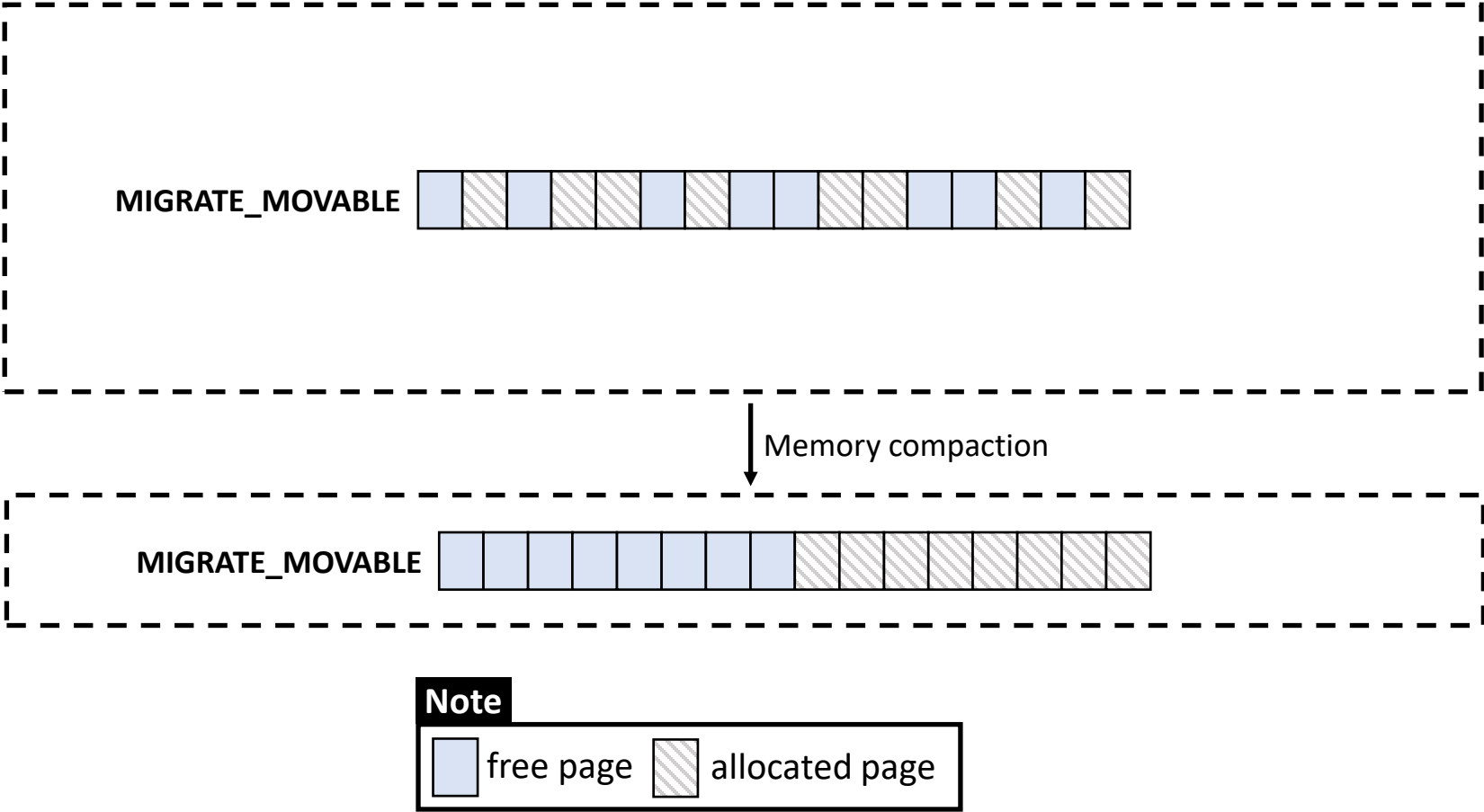
Physical memory defragmentation (Anti-fragmentation): Memory Migration (Mobility)



Memory migration delays memory fragmentation. It does not solve the problem

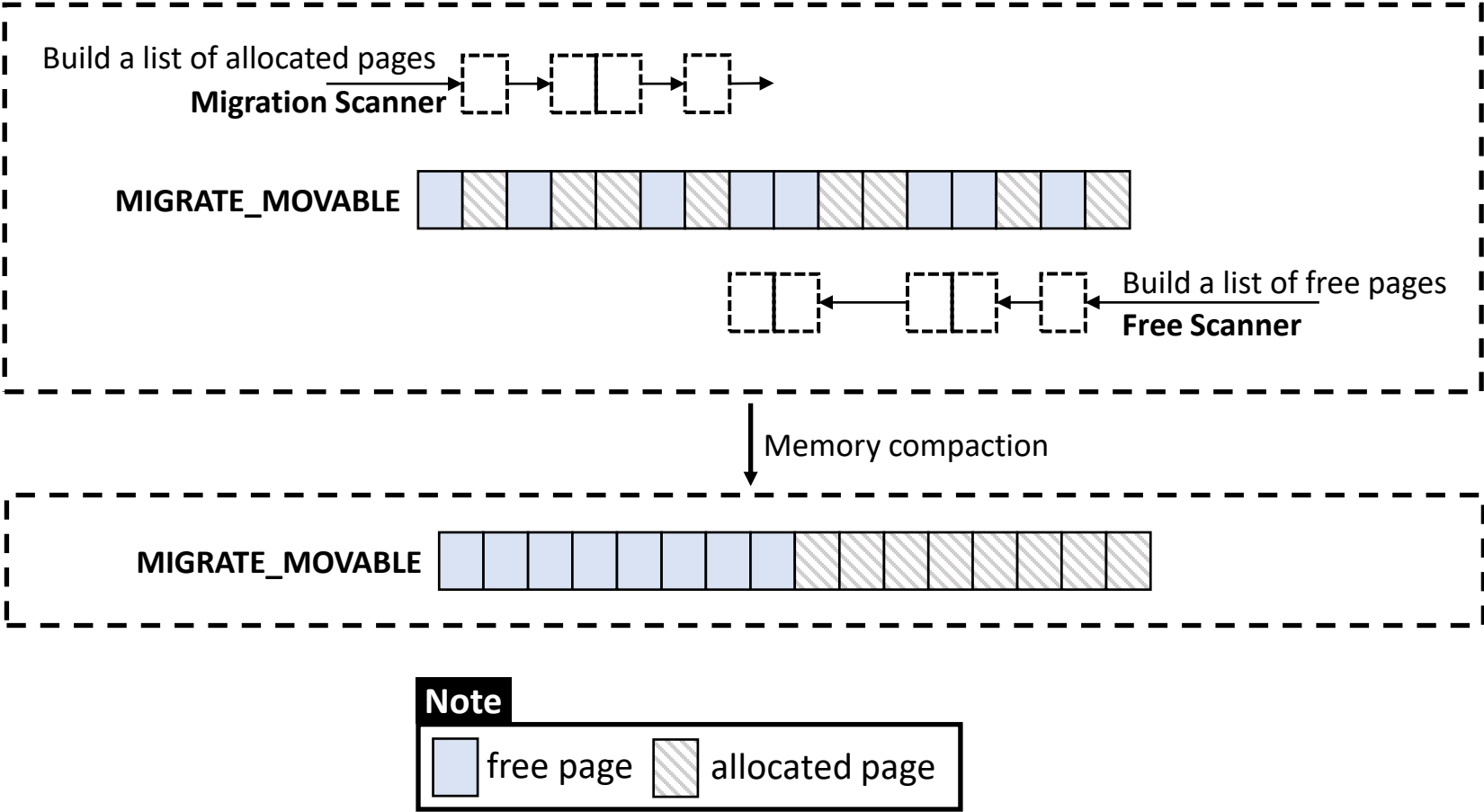
Physical memory defragmentation (Anti-fragmentation):

Memory compaction (1/2)



Physical memory defragmentation (Anti-fragmentation):

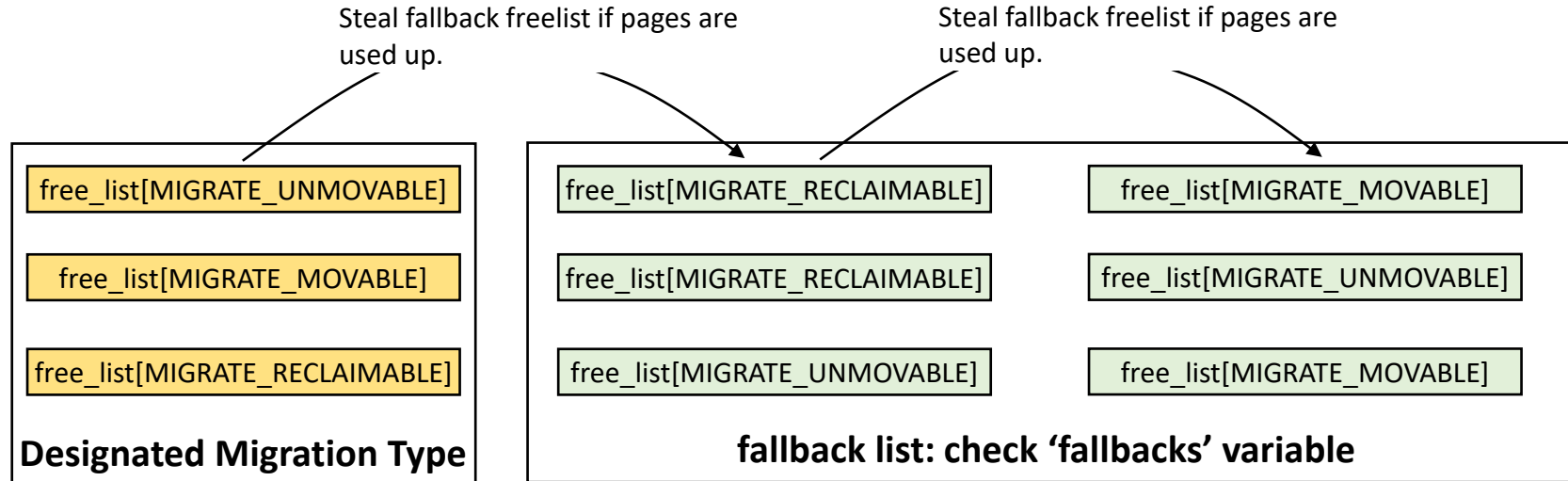
Memory compaction (2/2)



Physical memory defragmentation (anti-fragmentation)

1. Buddy system
2. Memory migration (mobility) ← detail discussion
3. Memory compaction

Anti-fragmentation: Memory migration (or Mobility)

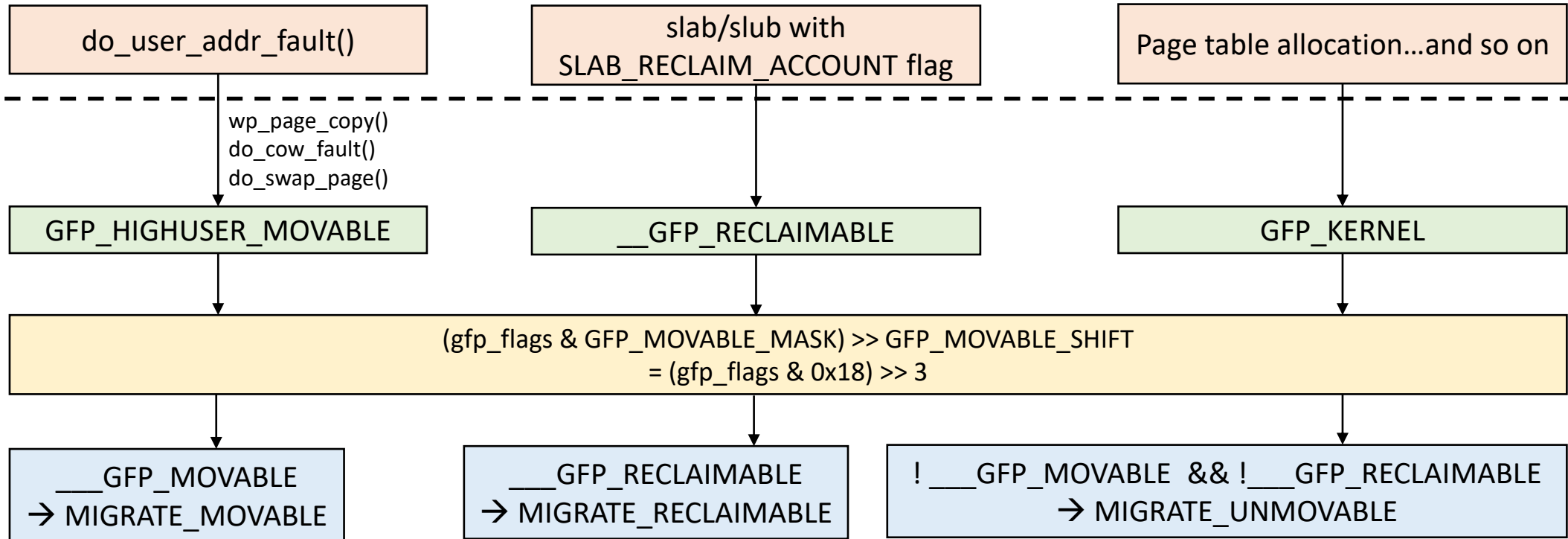


- `MIGRATE_UNMOVABLE`: Allocations of the core kernel
- `MIGRATE_MOVABLE` (`__GFP_MOVABLE`): Pages that belongs to userspace applications
- `MIGRATE_RECLAIMABLE` (`__GFP_RECLAIMABLE`): ~~File pages (Data mapped from files). Periodically reclaimed by the kswapd daemon~~
Slab/slub allocations that specify `SLAB_RECLAIM_ACCOUNT` and whose pages can be freed via shrinkers (kswapd): Check `include/linux/gfp.h`.
 - `SLAB_RECLAIM_ACCOUNT`:
 - mainly used by file system: data structure cache (fs/*).
 - radix tree: cache for 'struct radix_tree_node'

Page group concept: grouping pages with identical mobility (migration type)

Memory migration (or Mobility): Users

Memory Allocation



```
/*  
 * In case of changes, please don't forget to update  
 * include/trace/events/mmflags.h and tools/perf/builtin-kmem.c  
 */  
  
/* Plain integer GFP bitmasks. Do not use this directly. */  
#define __GFP_DMA 0x01u  
#define __GFP_HIGHMEM 0x02u  
#define __GFP_DMA32 0x04u  
#define __GFP_MOVABLE 0x08u  
#define __GFP_RECLAIMABLE 0x10u  
#define __GFP_HIGH 0x20u  
include/linux/gfp.h
```

```
(gdb) ptype /o enum migratetype  
type = enum migratetype {MIGRATE_UNMOVABLE, MIGRATE_MOVABLE,  
MIGRATE_RECLAIMABLE, MIGRATE_PCPTYPES, MIGRATE_HIGHATOMIC = 3,  
MIGRATE_TYPES}
```

* If page mobility is disabled, all pages are kept in `MIGRATE_UNMOVABLE`

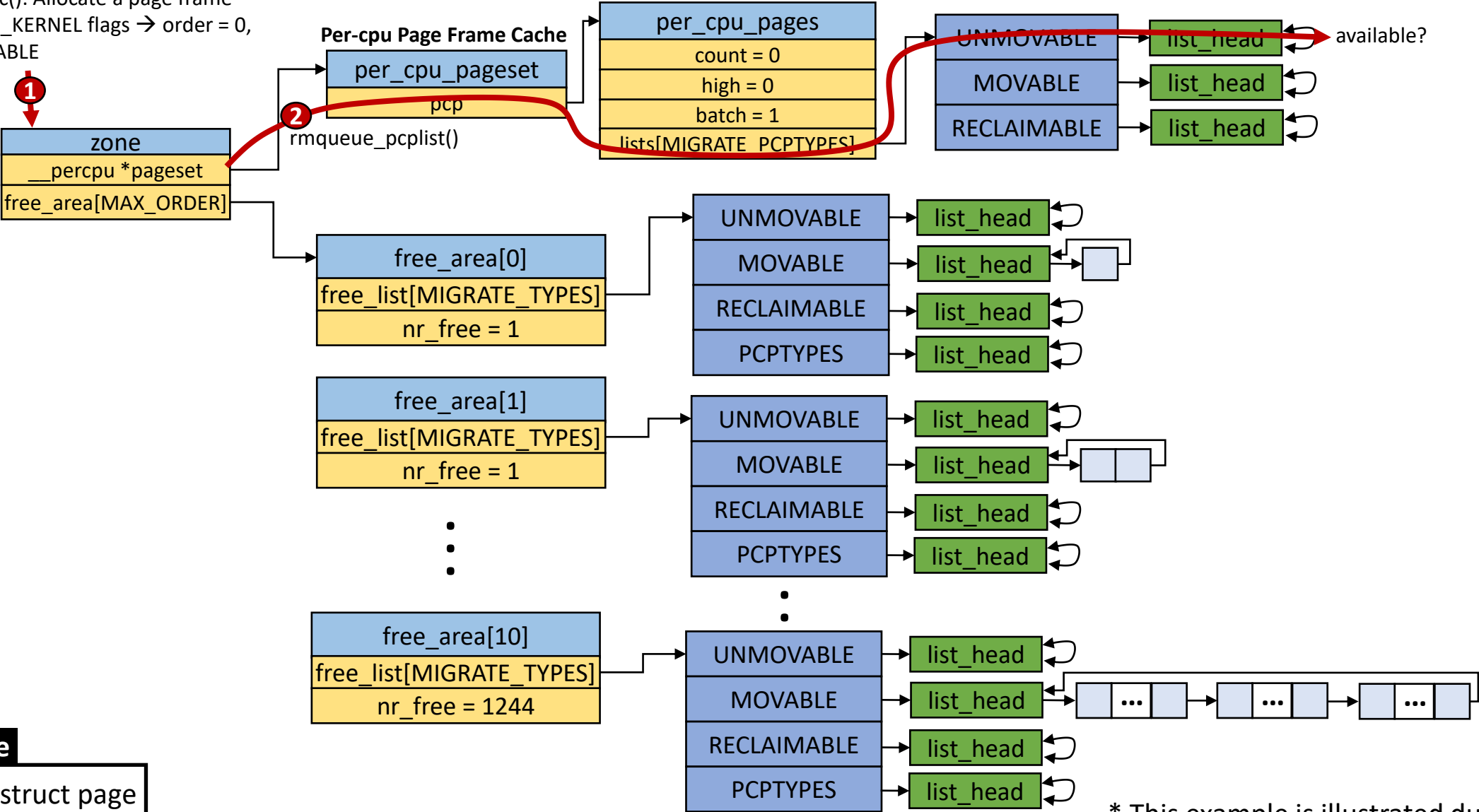
Page frame allocator: Physical memory defragmentation (anti-fragmentation)

Let's see an example:

1. How the migration works
2. How the buddy system works

Example: Scan available page frames from pcp

pud_alloc(): Allocate a page frame with GFP_KERNEL flags → order = 0, UNMOVABLE

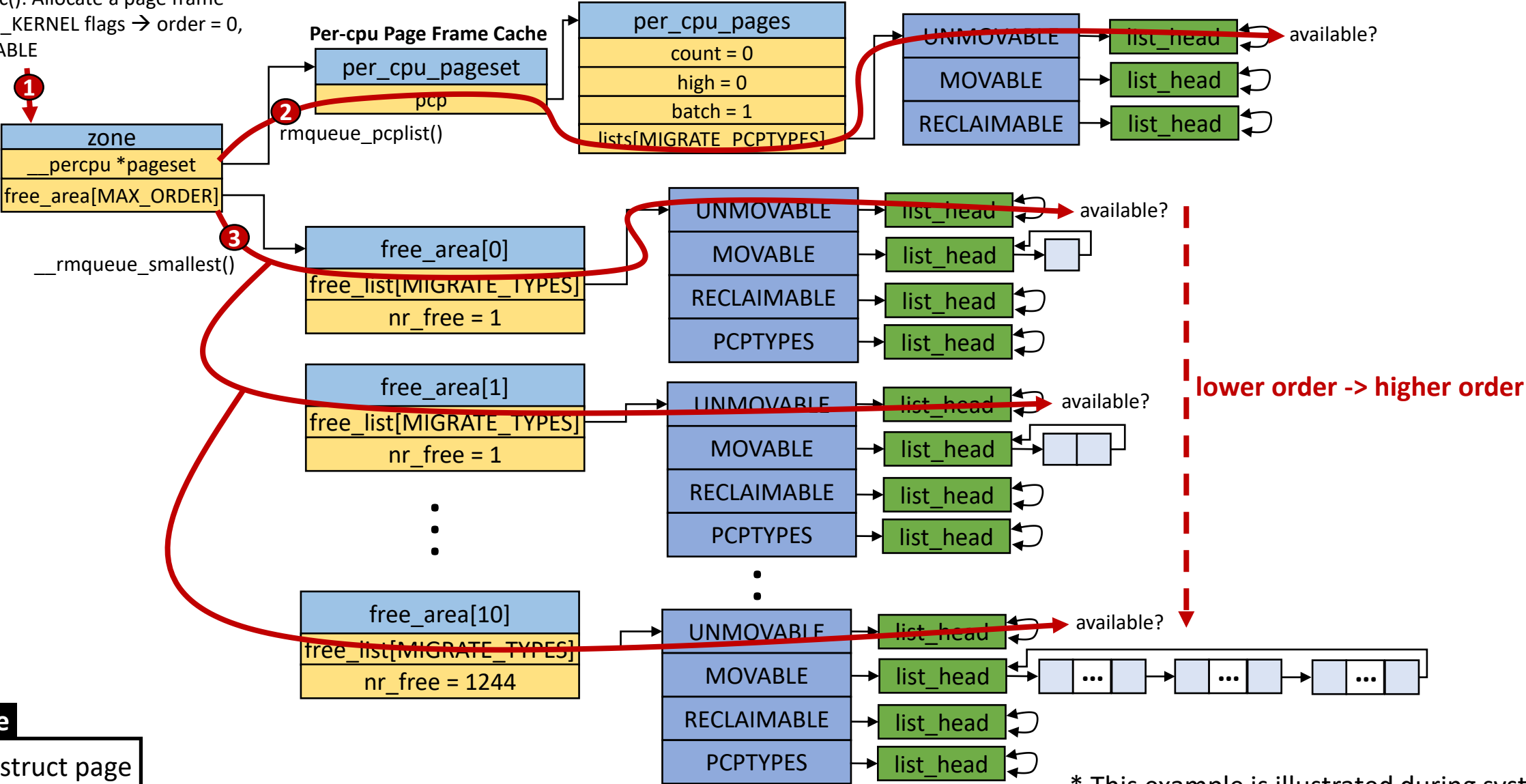


Note
[light blue box] struct page

* This example is illustrated during system init

Example: Scan available page frames from free_area

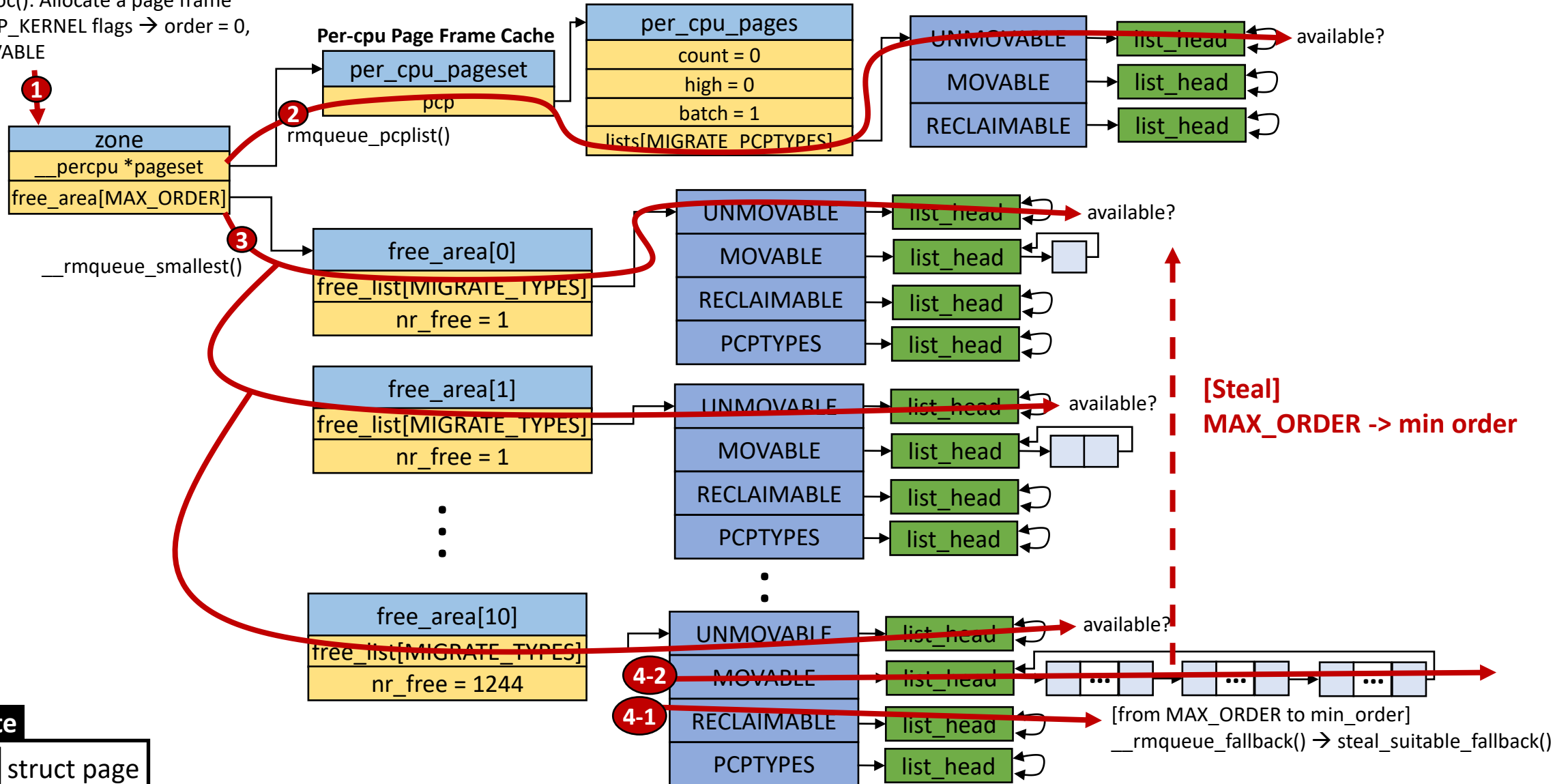
pud_alloc(): Allocate a page frame with GFP_KERNEL flags → order = 0, UNMOVABLE



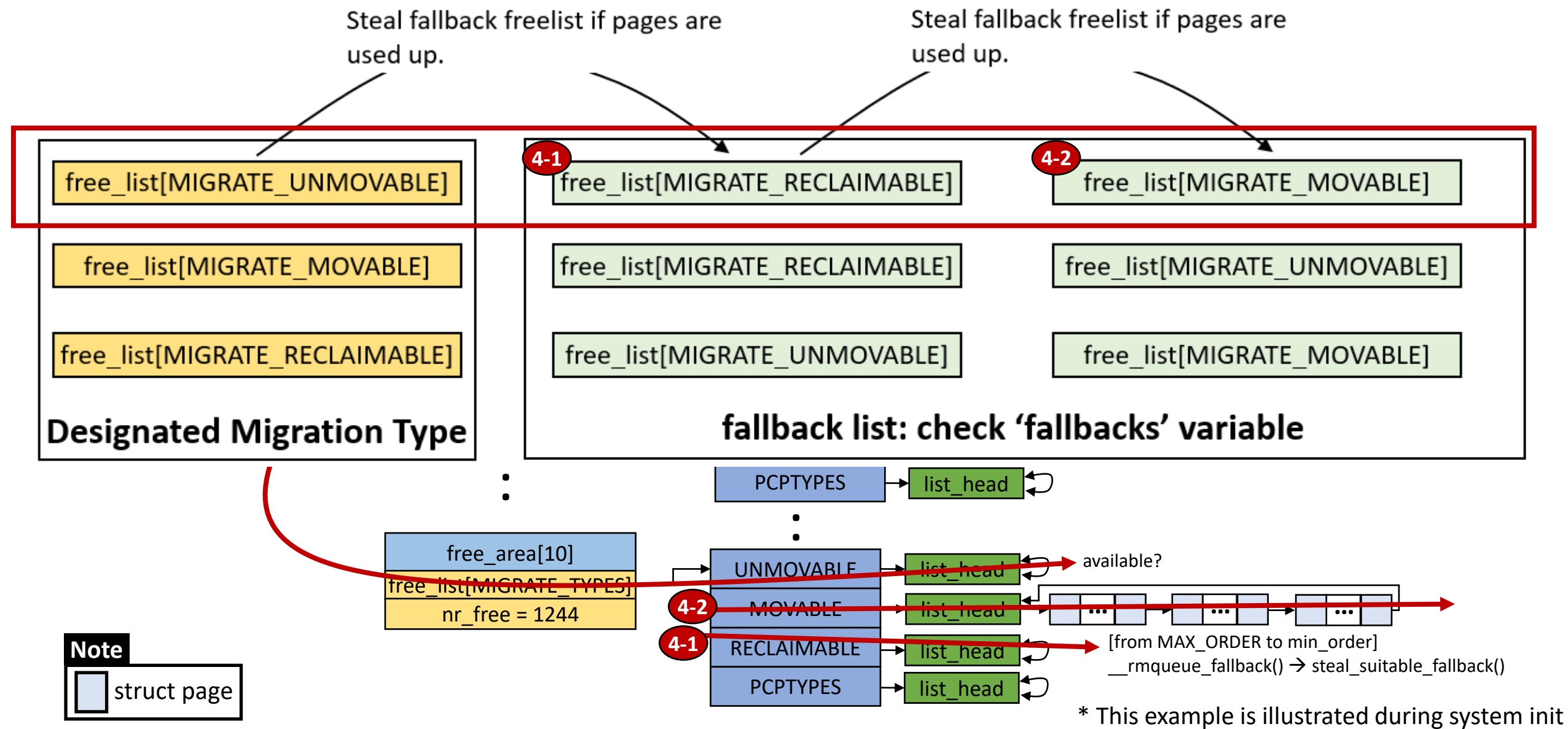
* This example is illustrated during system init

Example: No available page frames in pcp and free_area for specific migration type → steal from other migration type

pud_alloc(): Allocate a page frame with GFP_KERNEL flags → order = 0, UNMOVABLE

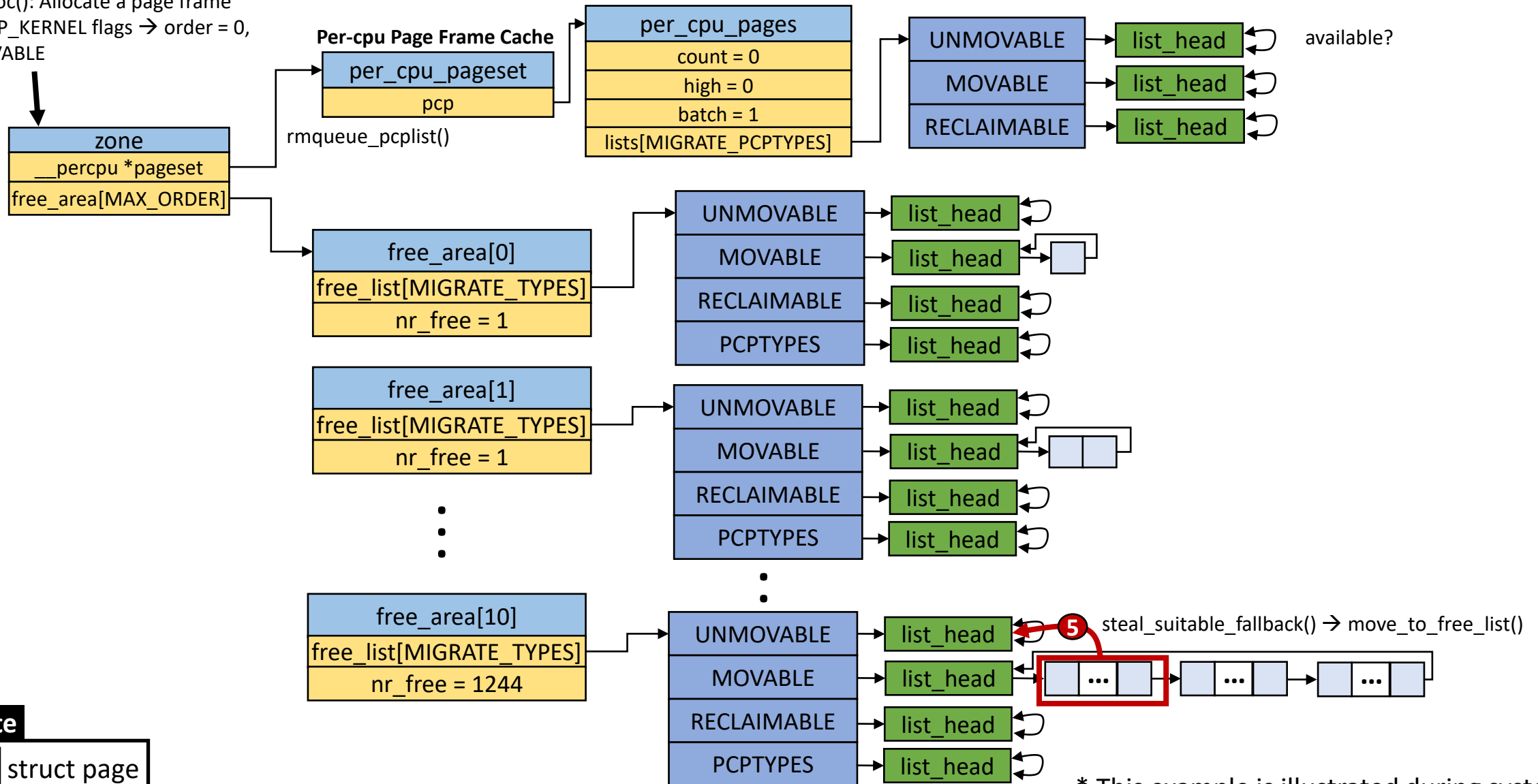


Example: No available page frames in pcp and free_area for specific migration type → steal from other migration type



Example: No available page frames in pcp and free_area for specific migration type → steal from other migration type

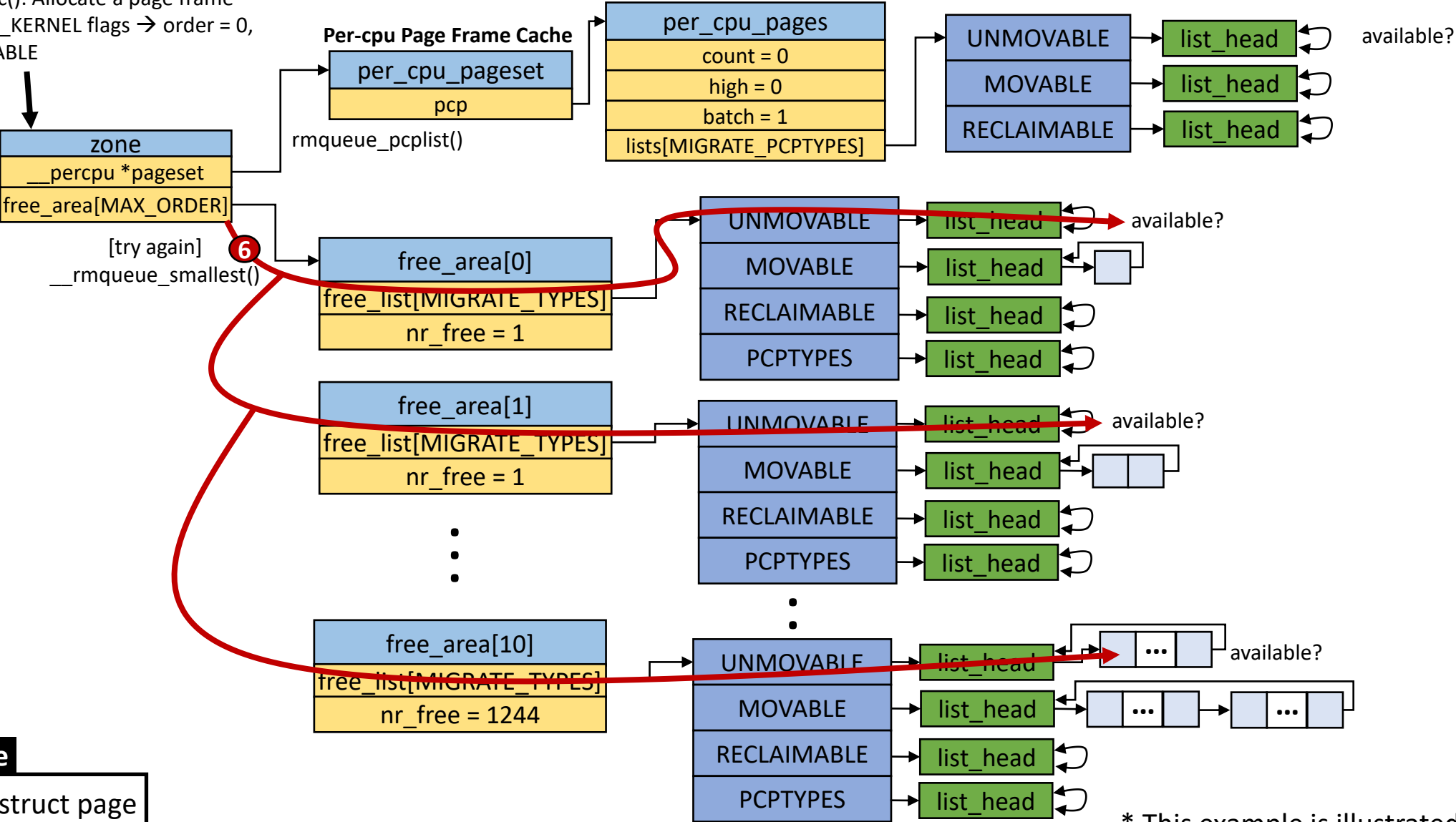
pud_alloc(): Allocate a page frame with GFP_KERNEL flags → order = 0, UNMOVABLE



* This example is illustrated during system init

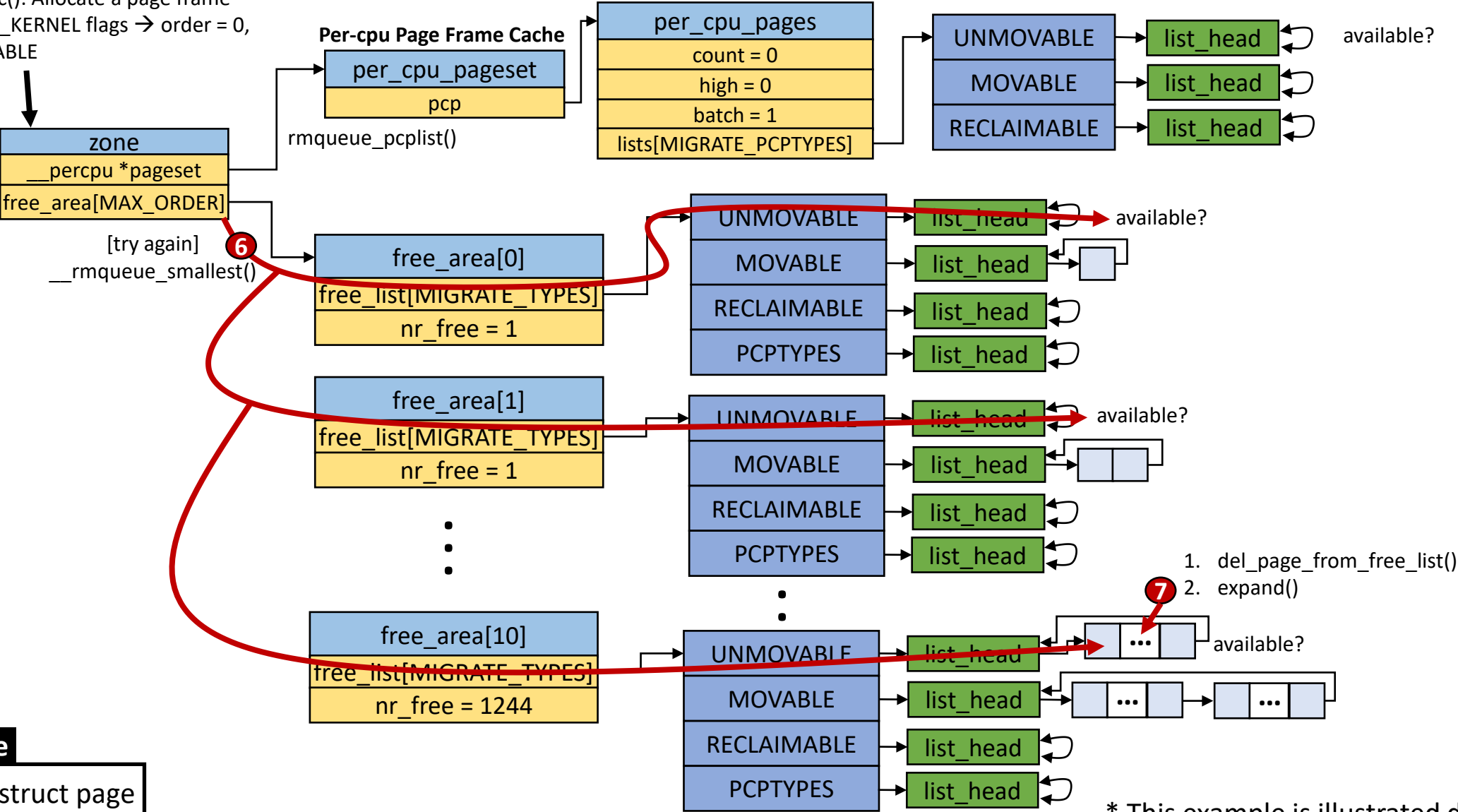
Example: Re-scan available page frames from free_list

pud_alloc(): Allocate a page frame with GFP_KERNEL flags → order = 0, UNMOVABLE

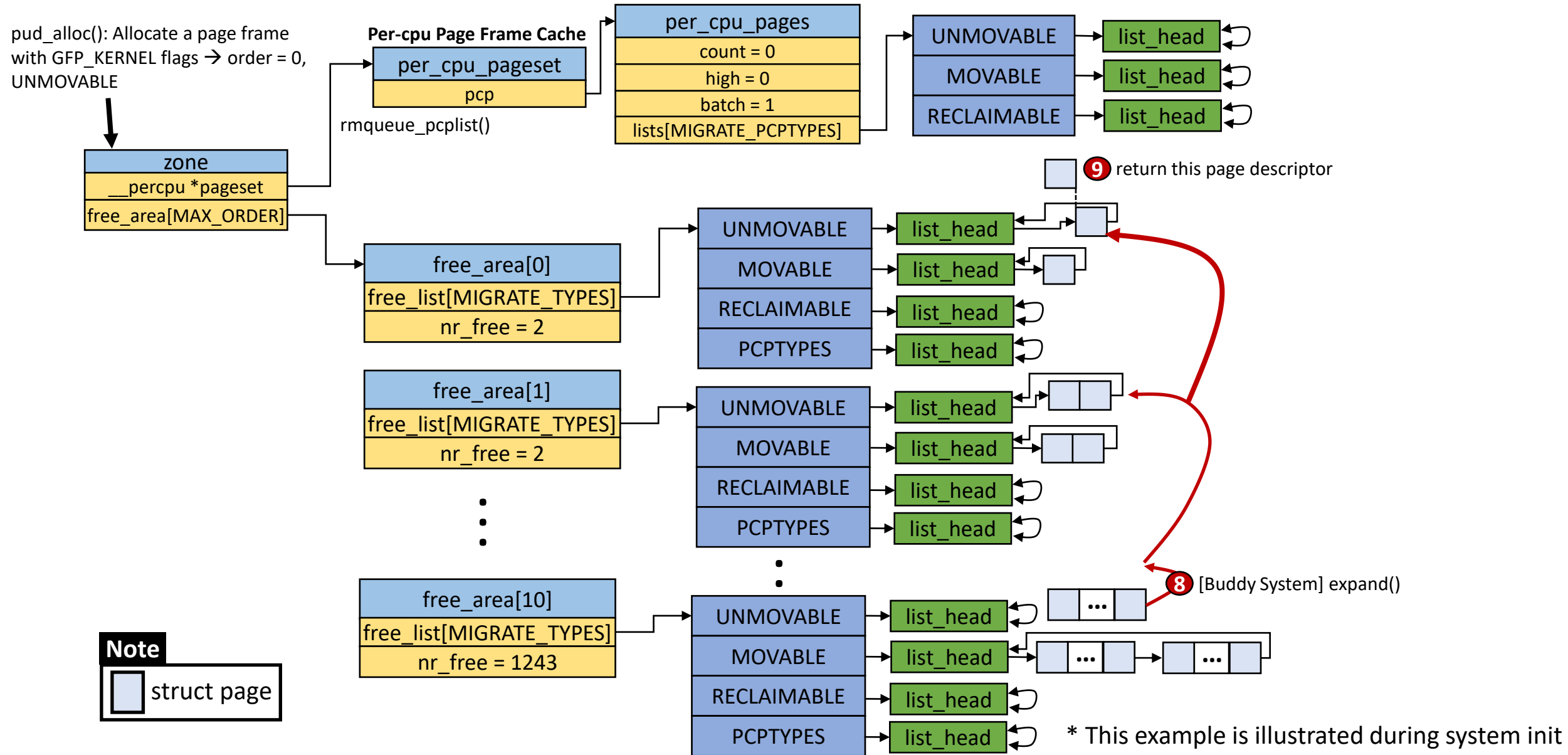


Example: Re-scan available page frames from free_list

pud_alloc(): Allocate a page frame with GFP_KERNEL flags → order = 0, UNMOVABLE



Example: Remove page frames from higher-order free_list and expand them to lower-order free_list



Example: Call Trace (1/2)

```
#0 set_pageblock_migratetype (page=page@entry=0xffffea0004000000,
migratetype=migratetype@entry=0)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
551
#1 0xffffffff810f44e4 in change_pageblock_range (migratetype=0,
start_order=<optimized out>, pageblock_page=0xffffea0004000000)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
2455
#2 steal_suitable_fallback (zone=zone@entry=0xffff8882403e34c0,
page=0xffffea0004000000, alloc_flags=alloc_flags@entry=257,
start_type=start_type@entry=0, whole_block=<optimized out>)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
2556
#3 0xffffffff810f5df3 in __rmqueue_fallback (alloc_flags=<optimized out>,
start_migratetype=<optimized out>, order=<optimized out>,
zone=<optimized out>)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
2847
#4 __rmqueue (alloc_flags=<optimized out>, migratetype=<optimized out>,
order=<optimized out>, zone=<optimized out>)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
2887
#5 rmqueue_bulk (order=0, alloc_flags=257, migratetype=0,
list=0xffff888237e1d1d0, count=<optimized out>, zone=0xffff8882403e34c0)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
2910
#6 __rmqueue_pcplist (list=0xffff888237e1d1d0, pcp=0xffff888237e1d1c0,
alloc_flags=257, migratetype=0, zone=0xffff8882403e34c0)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
3421
#7 rmqueue_pcplist (gfp_flags=256, alloc_flags=257, migratetype=0,
zone=0xffff8882403e34c0, preferred_zone=0xffff8882403e34c0)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
3449
#8 rmqueue (gfp_flags=256, migratetype=0, alloc_flags=257,
order=<optimized out>, zone=0xffff8882403e34c0,
preferred_zone=0xffff8882403e34c0)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
3477
#9 get_page_from_freelist (gfp_mask=gfp_mask@entry=256, order=order@entry=0,
alloc_flags=257, ac=ac@entry=0xffffffff81a03df8)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
3942
#10 0xffffffff810f6dc0 in __alloc_pages_nodemask (gfp_mask=256,
gfp_mask@entry=3520, order=order@entry=0, preferred_nid=<optimized out>,
nodemask=0x0 <fixed_percpu_data>)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
4995
#11 0xffffffff8110840f in alloc_pages_current (gfp=3520, order=order@entry=0)
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/mempolicy.c:1
394
```

breakpoints

```
(gdb) info b
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0xffffffff81001760 in emulate_vsyscall
                                           at /home/adrian/git-repo/gdb-
linux-real-mode/src/linux-5.11/arch/x86/entry/vsyscall/vsyscall_64.c:131
2        breakpoint keep y  0xffffffff810f3d80 in set_pageblock_migratetype
at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:551
stop only if migratetype == 0
breakpoint already hit 1 time
```

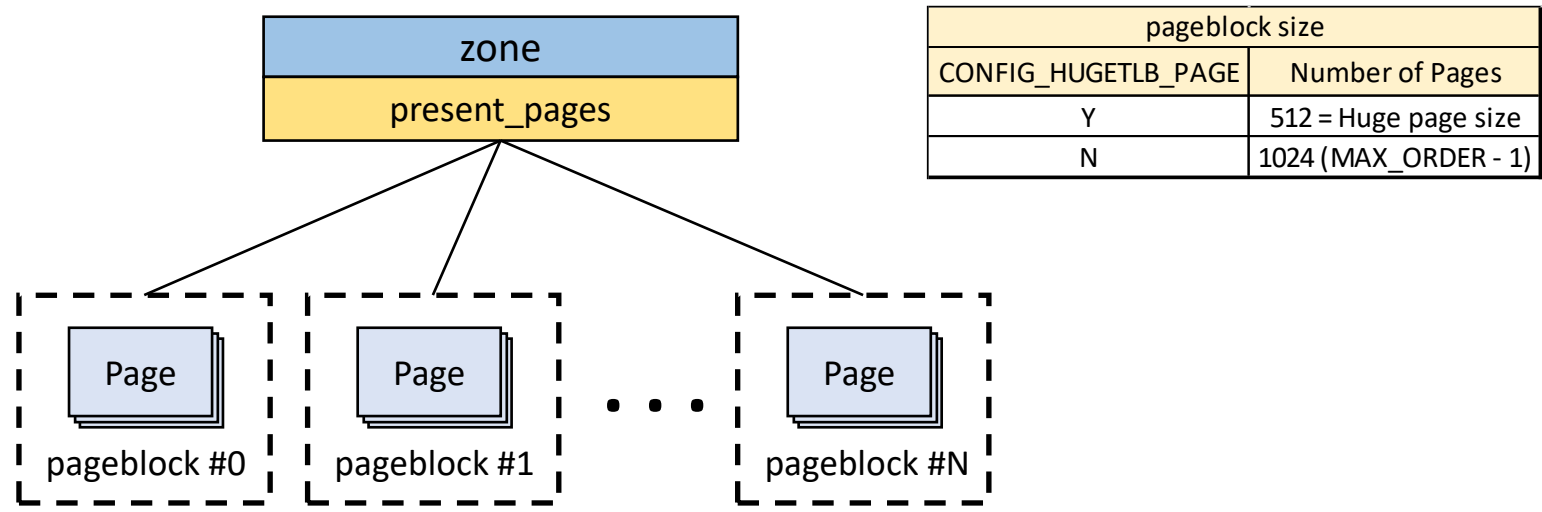
Example: Call Trace (2/2)

```
#12 0xffffffff810f295c in alloc_pages (order=order@entry=0,
    gfp_mask=<optimized out>)
--Type <RET> for more, q to quit, c to continue without paging--
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/include/linux/gfp.h:547
#13 __get_free_pages (gfp_mask=<optimized out>, order=order@entry=0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:5038
#14 0xffffffff810f2991 in get_zeroed_page (gfp_mask=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:5047
#15 0xffffffff810db988 in pud_alloc_one (addr=addr@entry=18446744071590079680,
    mm=0xffffffff81ab6cc0 <init_mm>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/include/asm-generic/pgalloc.h:166
#16 __pud_alloc (mm=0xffffffff81ab6cc0 <init_mm>,
    p4d=p4d@entry=0xffffffff81a0ac90 <init_top_pgt+3216>,
    address=address@entry=18446683600570023936)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/memory.c:4670
#17 0xffffffff81b075a5 in pud_alloc (mm=<optimized out>,
    address=18446683600570023936, p4d=0xffffffff81a0ac90 <init_top_pgt+3216>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/mm/init_64.c:1270
#18 preallocate_vmalloc_pages ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/mm/init_64.c:1269
#19 mem_init ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/mm/init_64.c:1308
#20 0xffffffff81af5d94 in mm_init ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/init/main.c:829
#21 start_kernel ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/init/main.c:905
#22 0xffffffff81af5496 in x86_64_start_reservations (
    real_mode_data=real_mode_data@entry=0x13a10 <bts_ctx+2576> <error: Cannot access memory at address 0x13a10>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/kernel/head64.c:525
#23 0xffffffff81af5519 in x86_64_start_kernel (
    real_mode_data=0x13a10 <bts_ctx+2576> <error: Cannot access memory at address 0x13a10>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/kernel/head64.c:506
#24 0xffffffff81000107 in secondary_startup_64 ()
```

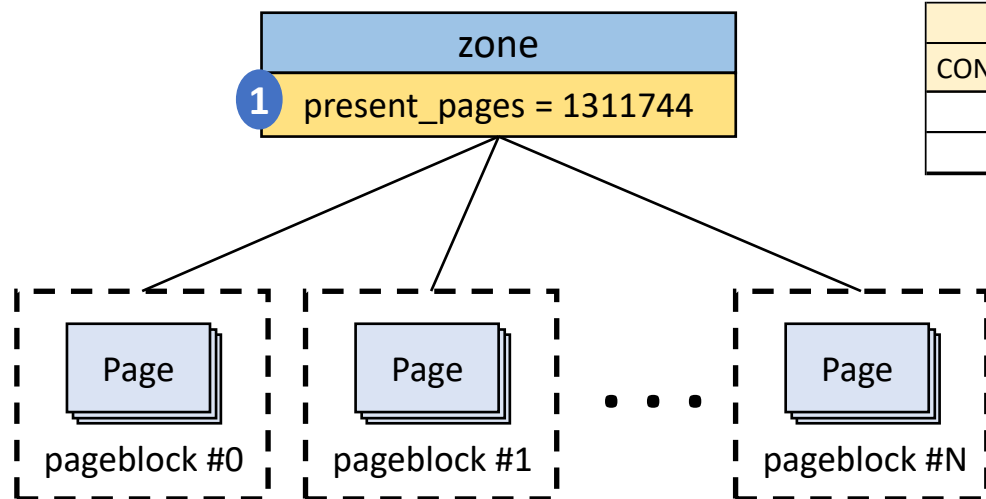
pageblock

1. How is a pageblock organized?
2. Relationship between pageblock and free_list (migrate list)
3. How to add pages to free_list (migrate list)?
 - pageblock, MAX_ORDER -1, or else?

pageblock



pageblock



pageblock size	
CONFIG_HUGETLB_PAGE	Number of Pages
Y	512 = Huge page size
N	1024 (MAX_ORDER - 1)

$$N = \text{round_up}(\text{present_pages} / \text{pageblock_size}) - 1$$

2

Example

$$\text{pageblocks} = \text{round_up}(1311744 / 512) = 2562$$

```
(gdb) p node_data[0]->node_zones[1]
$5 = {
  _watermark = {7040, 8800, 10560},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff8882403fa000,
  pageset = 0x21d28,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1048576,
  managed_pages = {
    counter = 1275118
  },
  spanned_pages = 1311744,
  present_pages = 1311744,
  name = 0xffffffff818f86b5 "Normal",
  ...
}
```

1

```
/ # cat /proc/pagetypeinfo
Page block order: 9
Pages per block: 512
```

Free pages count per migrate type at order	0	1	2	3	4	5	6	7	8	9	10
Node 0, zone DMA32, type Unmovable	0	1	0	1	0	0	1	0	1	0	0
Node 0, zone DMA32, type Movable	3	4	7	6	4	4	5	4	2	4	745
Node 0, zone DMA32, type Reclaimable	0	0	0	0	0	0	0	0	0	0	0
Node 0, zone DMA32, type HighAtomic	0	0	0	0	0	0	0	0	0	0	0
Node 0, zone Normal, type Unmovable	0	1	0	1	0	1	1	0	0	1	0
Node 0, zone Normal, type Movable	0	0	0	0	1	1	3	1	1	1	1235
Node 0, zone Normal, type Reclaimable	0	0	1	1	0	1	0	1	1	0	0
Node 0, zone Normal, type HighAtomic	0	0	0	0	0	0	0	0	0	0	0

Number of blocks type	Unmovable	Movable	Reclaimable	HighAtomic
Node 0, zone DMA32	1	1535	0	0
Node 0, zone Normal	16	2544	2	0

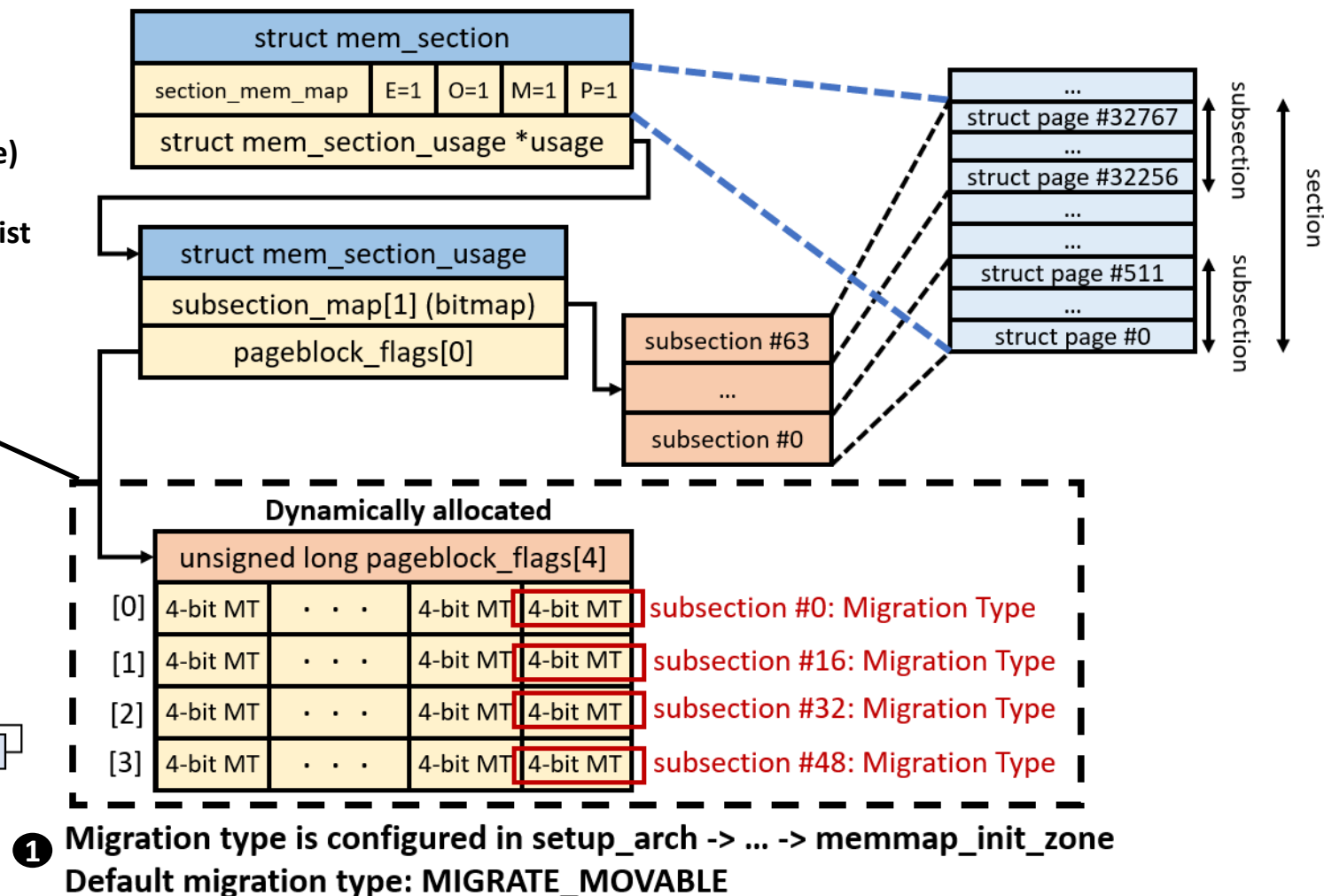
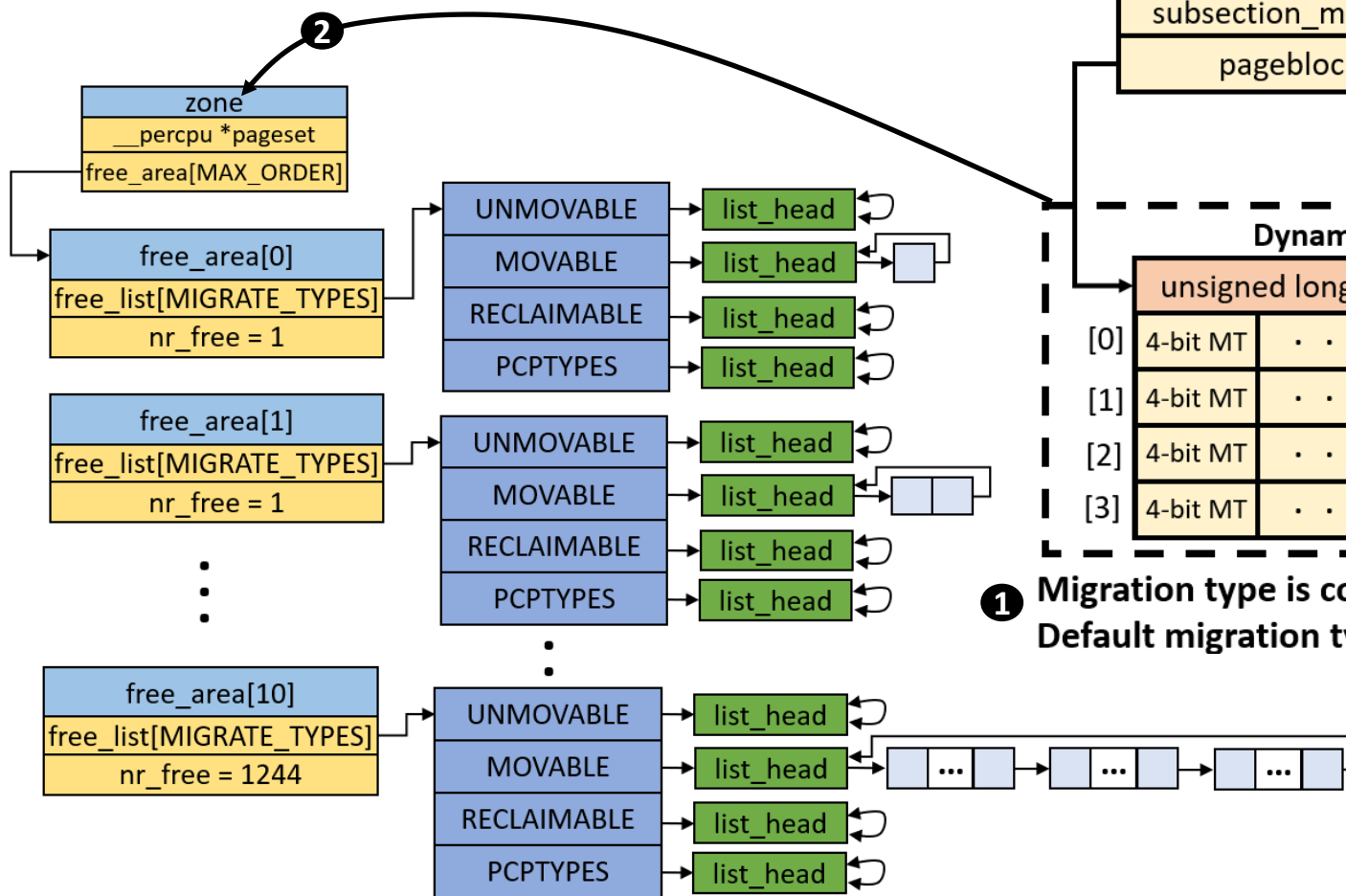
2

$$16 + 2544 + 2 = 2562$$

Relationship between pageblock and migrate list

Move available page to zone->free_area[] (use pageblock if possible)

- call path: mm_init -> ... -> __free_one_page -> add_to_free_list
- Two consecutive pageblocks can be emerged in order=10 free_list
 - ✓ Assume “pageblock = 512 pages (order = 9)”



Relationship between pageblock and migrate list

```
void __ref build_all_zonelists(pg_data_t *pgdat)
{
+-- 17 lines: unsigned long vm_total_pages;-----
    if (vm_total_pages < (pageblock_nr_pages * MIGRATE_TYPES))
        page_group_by_mobility_disabled = 1;
    else
        page_group_by_mobility_disabled = 0;

    pr_info("Built %u zonelists, mobility grouping %s. Total pages: %ld\n",
            nr_online_nodes,
            page_group_by_mobility_disabled ? "off" : "on",
            vm_total_pages);
#ifdef CONFIG_NUMA
    pr_info("Policy zone: %s\n", zone_names[policy_zone]);
#endif
}
mm/page_alloc.c 6050,1-8 679
```

Memory migration (page mobility) is disabled if number of pages is too low

pageblock – migration type during OS initialization?

During OS boots, all pageblocks are marked migration type 'MIGRATE_MOVABLE'

```
start_kernel [init/main.c]
|- setup_arch
... |- free_area_init_core
    |- pgdat_init_internals
        for (j = 0; j < MAX_NR_ZONES; j++) {
            |- zone_init_internals
                set managed_pages, nid, name, zone_pgdat
            |- zone_pcp_init
            |- init_currently_empty_zone
            |- zone_init_free_lists
                init all free_lists
        ① |- memmap_init
            for_each_mem_pfn_range(..) {
                |- memmap_init_zone
                    for (pfn = start_pfn; pfn < end_pfn;)
                        |- __init_single_page
                            1. Init fields of struct page
                            2. Set pageblock migration type
                                'MIGRATE_MOVABLE'
                        }
            }
    }
|- mm_init
    |- mem_init
        |- memblock_free_all
        |- free_low_memory_core_early
        for_each_reserved_mem_range(...)
            |- reserve_bootmem_region
        for_each_free_mem_range(...)
            |- __free_memory_core
            |- __free_pages_memory
            |- memblock_free_pages
            |- __free_pages_core
                1. Clear flag 'PG_reserved' of each
                    page struct
                2. Add number of pages to
                    zone.managed_pages
            |- __free_pages_ok
            |- free_one_page
            |- __free_one_page
            |- add_to_free_list
```

```
void __meminit __weak memmap_init(unsigned long size, int nid,
                                   unsigned long zone,
                                   unsigned long range_start_pfn)
{
+-- 4 lines: unsigned long start_pfn, end_pfn;-----
    for_each_mem_pfn_range(i, nid, &start_pfn, &end_pfn, NULL) {
        start_pfn = clamp(start_pfn, range_start_pfn, range_end_pfn);
        end_pfn = clamp(end_pfn, range_start_pfn, range_end_pfn);

        if (end_pfn > start_pfn) {
            size = end_pfn - start_pfn;
            ② memmap_init_zone(size, nid, zone, start_pfn, range_end_pfn,
                                MEMINIT_EARLY, NULL, MIGRATE_MOVABLE);
        }
    }
}
```

```
void __meminit memmap_init_zone(unsigned long size, int nid, unsigned long zone,
                                 unsigned long start_pfn, unsigned long zone_end_pfn,
                                 enum meminit_context context,
                                 struct vmem_altmap *altmap, int migratetype)
{
+-- 24 lines: unsigned long pfn, end_pfn = start_pfn + size;-----
    for (pfn = start_pfn; pfn < end_pfn; ) {
+-- 16 lines: There can be holes in boot-time mem_map[]s handed to this-----
        /*
         * Usually, we want to mark the pageblock MIGRATE_MOVABLE,
         * such that unmovable allocations won't be scattered all
         * over the place during system boot.
         */
        ③ if (IS_ALIGNED(pfn, pageblock_nr_pages)) {
            set_pageblock_migratetype(page, migratetype);
            cond_resched();
        }
        pfn++;
    }
}
```

How to add pages to free_list (migrate list)?

Which method?

- pageblock?
- MAX_ORDER -1?
- else?

```
start_kernel [init/main.c]
|- setup_arch
... |- free_area_init_core
    |- pgdat_init_internals
      for (j = 0; j < MAX_NR_ZONES; j++) {
        |- zone_init_internals
          set managed_pages, nid, name, zone_pgdat
          |- zone_pcp_init
          |- init_currently_empty_zone
          |- zone_init_free_lists
            init all free_lists
          |- memmap_init
            for_each_mem_pfn_range(..) {
              |- memmap_init_zone
                for (pfn = start_pfn; pfn < end_pfn;)
                  |- __init_single_page
                    1. Init fields of struct page
                    2. Set pageblock migration type
                      'MIGRATE_MOVABLE'
                }
            }
        }
    }
|- mm_init
  |- mem_init
    |- memblock_free_all
    |- free_low_memory_core_early
    for_each_reserved_mem_range(...)
      |- reserve bootmem region
    for_each_free_mem_range(...)
      |- __free_memory_core
      |- __free_pages_memory
      |- memblock_free_pages
      |- __free_pages_core
        1. Clear flag 'PG_reserved' of each
           page struct
        2. Add number of pages to
           zone.managed_pages
      |- __free_pages_ok
      |- free_one_page
        |- __free_one_page
        |- add_to_free_list
```

How to add pages to free_list (migrate list)?

```
start_kernel [init/main.c]
|- setup_arch
... |- free_area_init_core
    |- pgdat_init_internals
      for (j = 0; j < MAX_NR_ZONES; j++) {
        |- zone_init_internals
          set managed_pages, nid, name, zone_pgdat
        |- zone_pcp_init
        |- init_currently_empty_zone
        |- zone_init_free_lists
          init all free_lists
        |- memmap_init
          for_each_mem_pfn_range(..) {
            |- memmap_init_zone
              for (pfn = start_pfn; pfn < end_pfn;)
                |- __init_single_page
                  1. Init fields of struct page
                  2. Set pageblock migration type
                     'MIGRATE_MOVABLE'
            }
          }
        }
|- mm_init
  |- mem_init
    |- memblock_free_all
    |- free_low_memory_core_early
    for_each_reserved_mem_range(...)
      |- reserve bootmem region
    for_each_free_mem_range(...)
      |- __free_memory_core
      |- __free_pages_memory
      |- memblock_free_pages
      |- __free_pages_core
        1. Clear flag 'PG_reserved' of each
           page struct
        2. Add number of pages to
           zone.managed_pages
      |- __free_pages_ok
      |- free_one_page
        |- __free_one_page
        |- add_to_free_list
```

Which method?

- pageblock?
- MAX_ORDER - 1?
- else?

Principle

- Zone->free_area[MAX_ORDER]: From highest order to lowest order based on start pfn (iteratively merge lower-order pages if possible: see __free_one_page)

```
static void __init __free_pages_memory(unsigned long start, unsigned long end)
{
    int order;

    while (start < end) {
        order = min(MAX_ORDER - 1UL, __ffs(start));

        while (start + (1UL << order) > end)
            order--;

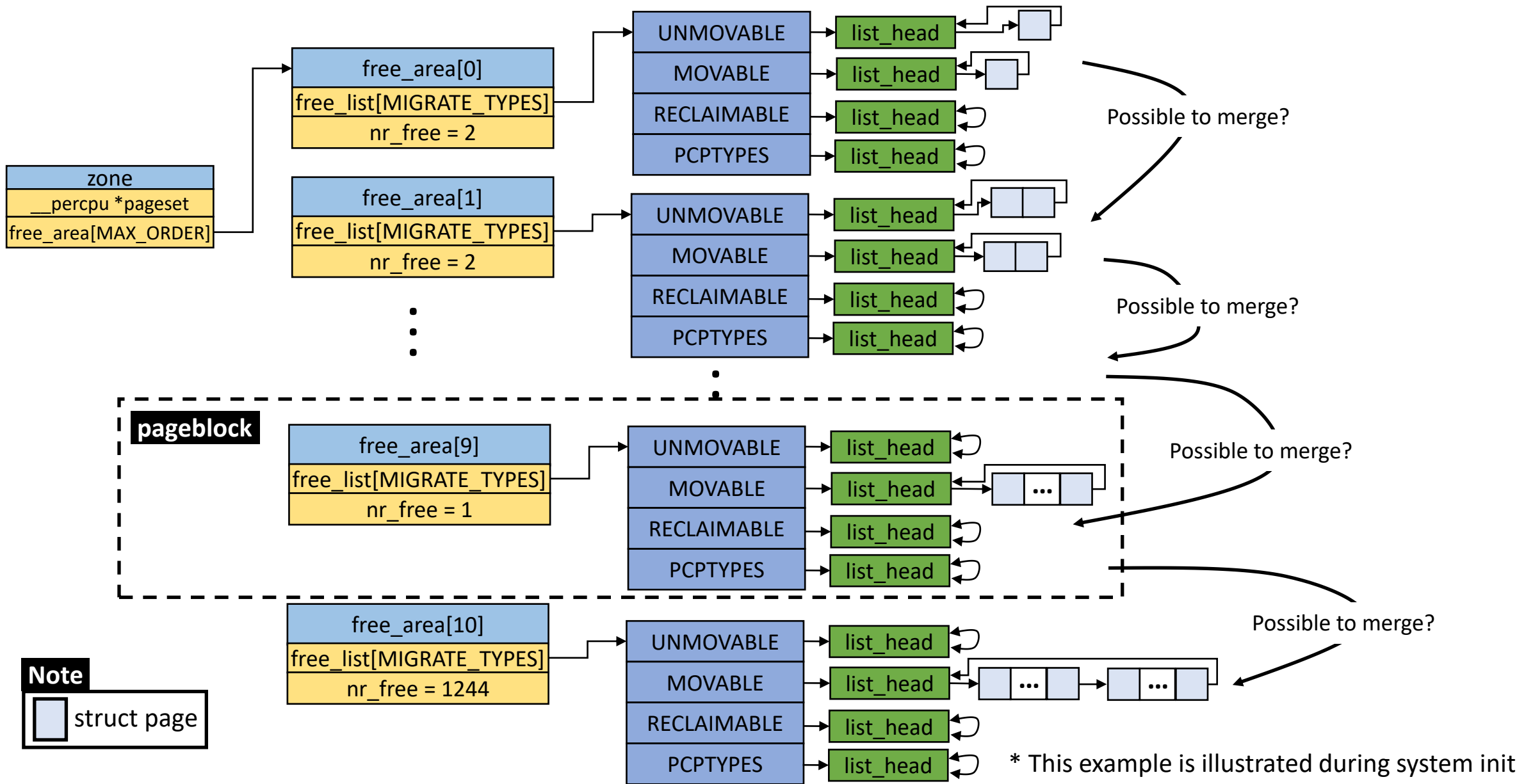
        memblock_free_pages(pfn_to_page(start), start, order);

        start += (1UL << order);
    }
}
mm/memblock.c 1972,1 9
```

```
static inline void __free_one_page(struct page *page,
                                   unsigned long pfn,
                                   struct zone *zone, unsigned int order,
                                   int migratetype, fpi_t fpi_flags)
{
    +-- 7 lines: struct capture_control *capc = task_capc(zone);-----
               max_order = min_t(unsigned int, MAX_ORDER - 1, pageblock_order);
    +-- 81 lines: VM_BUG_ON(!zone_is_initialized(zone));-----
}

```

__free_one_page(): continue to merge



dmesh output: total pages

dmesg output: total pages

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

```
/ # dmesg | grep -i total | grep page
On node 0 totalpages: 2097022
On node 1 totalpages: 2097152
Built 2 zonelists, mobility grouping on. Total pages: 4128619
```

$$2097022 + 2097152 = 4194174$$



$$4194174 \neq 4128619$$



Why?

dmesg output: total pages

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0 loglevel=8 nokaslr

```
/ # dmesg | grep -i total | grep page
On node 0 totalpages: 2097022
On node 1 totalpages: 2097152
Built 2 zonelists, mobility grouping on. Total pages: 4128619
```

$$2097022 + 2097152 = 4194174$$



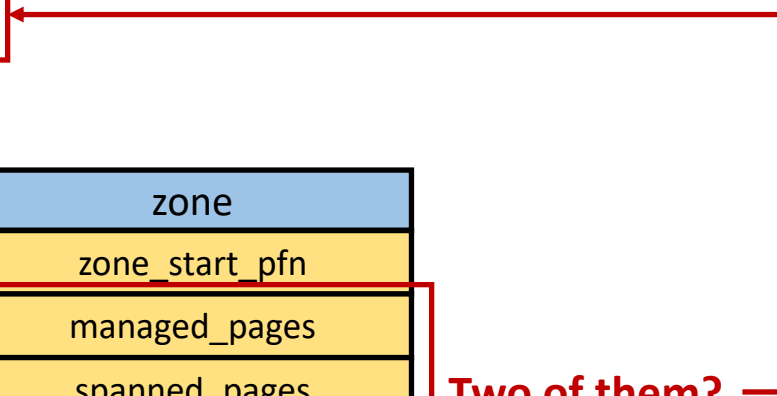
4194174 != 4128619



Why?

zone
zone_start_pfn
managed_pages
spanned_pages
present_pages
free_area[MAX_ORDER]

Two of them?



dmesg output: total pages

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

```
/ # dmesg | grep -i total | grep page
On node 0 totalpages: 2097022
On node 1 totalpages: 2097152
Built 2 zonelists, mobility grouping on. Total pages: 4128619
```

$$2097022 + 2097152 = 4194174$$



$$4194174 \neq 4128619$$



Why?



```
/ # dmesg | grep -i total | grep page
On node 0 totalpages: 2097022
On node 1 totalpages: 2097152
Built 2 zonelists, mobility grouping on. Total pages: 4128619
```

Total present_pages

Total managed_pages

Total present_pages & total managed_pages

```
/ # dmesg | grep -i total | grep page
On node 0 totalpages: 2097022
On node 1 totalpages: 2097152
Built 2 zonelists, mobility grouping on.  Total pages: 4128619
```

Total present_pages

Total managed_pages

```
start kernel [init/main.c]
|- setup_arch [arch/x86/kernel/setup.c]
|- x86_init.paging.pagetable_init
|- sparse_init
|- zone_sizes_init
|- free_area_init
   print zone info
   for_each_mem_pfn_range(..)
       subsection_map_init
   for_each_online_node(nid)
       |- free_area_init_node(nid);
          |- calculate_node_totalpages
             set zone_start_pfn, spanned_pages
             present_pages, node_spanned_pages
             and node_present_pages.
          |- free_area_init_core
             |- zone_init_internals
                set zone.managed_pages
|- build_all_zonelists [mm/page_alloc.c]
```

Total present_pages

Total managed_pages

calculate_node_totalpages

- Calculate zone.spanned_pages and zone.present_pages for each zone
- Sum each zone.present_pages to get total present_pages
- Print "On node 0 totalpages:...." message

free_area_init_core & zone_init_internals

- Calculate managed_pages and set zone.managed_pages

build_all_zonelists

- Sum all zone.managed_pages to get total managed_pages and print "Built %u zonelists, ..." message

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

Total present_pages: breakdown

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

Let's focus on "node 0"

```
Early memory node ranges
node 0: [mem 0x0000000000001000-0x0000000000009efff]
node 0: [mem 0x0000000000100000-0x000000000bffdffff]
node 0: [mem 0x0000000100000000-0x000000023fffffff]
node 1: [mem 0x0000000240000000-0x000000043fffffff]
Zeroed struct page in unavailable ranges: 130 pages
Initmem setup node 0 [mem 0x0000000000001000-0x000000023fffffff]
On node 0 totalpages: 2097022
  DMA32 zone: 12286 pages used for memmap
  DMA32 zone: 21 pages reserved
  DMA32 zone: 786302 pages, LIFO batch:63
  Normal zone: 20480 pages used for memmap
  Normal zone: 1310720 pages, LIFO batch:63
Initmem setup node 1 [mem 0x0000000240000000-0x000000043fffffff]
On node 1 totalpages: 2097152
  Normal zone: 32768 pages used for memmap
  Normal zone: 2097152 pages, LIFO batch:63
```

Total present_pages: breakdown

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

```
Early memory node ranges
 node 0: [mem 0x00000000000001000-0x00000000000009efff]
 node 0: [mem 0x000000000000100000-0x000000000bffdffff]
 node 0: [mem 0x00000000100000000-0x0000000023fffffff]
 node 1: [mem 0x00000000240000000-0x0000000043fffffff]
Zeroed struct page in unavailable ranges: 130 pages
Initmem setup node 0 [mem 0x00000000000001000-0x0000000023fffffff]
On node 0 totalpages: 2097022
  DMA32 zone: 12286 pages used for memmap
  DMA32 zone: 21 pages reserved
  DMA32 zone: 786302 pages, LIFO batch:63
  Normal zone: 20480 pages used for memmap
  Normal zone: 1310720 pages, LIFO batch:63
Initmem setup node 1 [mem 0x00000000240000000-0x0000000043fffffff]
On node 1 totalpages: 2097152
  Normal zone: 32768 pages used for memmap
  Normal zone: 2097152 pages, LIFO batch:63
```

$$\text{Total present_pages} = 786302 + 1310720 = 2097022$$

Total present_pages: breakdown

```
BIOS-e820: [mem 0x0000000000000000-0x0000000000009fbfff] usable
BIOS-e820: [mem 0x0000000000009fc00-0x0000000000009ffff] reserved
BIOS-e820: [mem 0x000000000000f0000-0x000000000000fffff] reserved
BIOS-e820: [mem 0x00000000000100000-0x000000000000bffdffff] usable
BIOS-e820: [mem 0x00000000000bffe0000-0x00000000000bffffff] reserved
BIOS-e820: [mem 0x0000000000fffc0000-0x0000000000ffffff] reserved
BIOS-e820: [mem 0x00000000100000000-0x0000000043ffffff] usable
```

```
Early memory node ranges
 node 0: [mem 0x00000000000001000-0x0000000000009efff]
 node 0: [mem 0x00000000000100000-0x000000000000bffdffff]
 node 0: [mem 0x00000000100000000-0x00000000023ffffff]
 node 1: [mem 0x00000000240000000-0x00000000043ffffff]
Zeroed struct page in unavailable ranges: 130 pages
Initmem setup node 0 [mem 0x00000000000001000-0x00000000023ffffff]
On node 0 totalpages: 2097022
 DMA32 zone: 12286 pages used for memmap ←
 DMA32 zone: 21 pages reserved ←
 DMA32 zone: 786302 pages, LIFO batch:63
 Normal zone: 20480 pages used for memmap ←
 Normal zone: 1310720 pages, LIFO batch:63
Initmem setup node 1 [mem 0x00000000240000000-0x00000000043ffffff]
On node 1 totalpages: 2097152
 Normal zone: 32768 pages used for memmap ←
 Normal zone: 2097152 pages, LIFO batch:63
```

Questions

1. What does it mean about "... %lu pages used for memmap"?
 - Number of page structs to address a zone
2. What does it mean about "... %lu pages reserved"?
 - Number pages reserved for DMA zone (check global variable 'dma_reserve')
3. Why aren't the above-mentioned pages accumulated in present_pages?
 - Pre-calculate page space requirement: Will consume total present_pages

* Based on kernel 5.11 (x86_64) – QEMU
* 2-socket CPUs (4 cores/socket)
* 16GB memory
* earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

Total present_pages: breakdown

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0 loglevel=8 nokaslr

```
Early memory node ranges
node 0: [mem 0x0000000000001000-0x0000000000009efff]
node 0: [mem 0x0000000000100000-0x00000000bffdffff]
node 0: [mem 0x0000000010000000-0x0000000023ffffff]
node 1: [mem 0x0000000024000000-0x0000000043ffffff]
Zeroed struct page in unavailable ranges: 130 pages
Initmem setup node 0 [mem 0x0000000000001000-0x0000000023ffffff]
On node 0 totalpages: 2097022
DMA32 zone: 12286 pages used for memmap
DMA32 zone: 21 pages reserved
DMA32 zone: 786302 pages, LIFO batch:63
Normal zone: 20480 pages used for memmap
Normal zone: 1310720 pages, LIFO batch:63
Initmem setup node 1 [mem 0x0000000024000000-0x0000000043ffffff]
On node 1 totalpages: 2097152
Normal zone: 32768 pages used for memmap
Normal zone: 2097152 pages, LIFO batch:63
```

Number of page structs to address a zone

- `sizeof(struct page) = 64`
- $12286 * 4096 / 64 = 786304$ (page struct)

Total present_pages: breakdown

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

```
Early memory node ranges
node 0: [mem 0x0000000000001000-0x0000000000009efff]
node 0: [mem 0x0000000000010000-0x00000000bffdffff]
node 0: [mem 0x0000000100000000-0x0000000023ffffff]
node 1: [mem 0x0000000240000000-0x0000000043ffffff]
Zeroed struct page in unavailable ranges: 130 pages
Initmem setup node 0 [mem 0x0000000000001000-0x0000000023ffffff]
On node 0 totalpages: 2097022
DMA32 zone: 12286 pages used for memmap
DMA32 zone: 21 pages reserved
DMA32 zone: 786302 pages, LIFO batch:63
Normal zone: 20480 pages used for memmap
Normal zone: 1310720 pages, LIFO batch:63
Initmem setup node 1 [mem 0x0000000240000000-0x0000000043ffffff]
On node 1 totalpages: 2097152
Normal zone: 32768 pages used for memmap
Normal zone: 2097152 pages, LIFO batch:63
```

match: the difference is the page alignment

Number of page structs to address a zone

- $\text{sizeof}(\text{struct page}) = 64$
- $12286 * 4096 / 64 = 786304 \text{ (page struct)}$

Total present_pages: breakdown

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0 loglevel=8 nokaslr

```
Early memory node ranges
node 0: [mem 0x0000000000001000-0x000000000009efff]
node 0: [mem 0x0000000000100000-0x00000000bfffdfbf]
node 0: [mem 0x0000000010000000-0x0000000023ffffff]
node 1: [mem 0x0000000024000000-0x0000000043ffffff]
Zeroed struct page in unavailable ranges: 130 pages
Initmem setup node 0 [mem 0x0000000000001000-0x0000000023ffffff]
On node 0 totalpages: 2097022
DMA32 zone: 12286 pages used for memmap
DMA32 zone: 21 pages reserved
DMA32 zone: 786302 pages, LIFO batch:63
Normal zone: 20480 pages used for memmap
Normal zone: 1310720 pages, LIFO batch:63
Initmem setup node 1 [mem 0x0000000024000000-0x0000000043ffffff]
On node 1 totalpages: 2097152
Normal zone: 32768 pages used for memmap
Normal zone: 2097152 pages, LIFO batch:63
```

sparse case

```
static unsigned long __init calc_memmap_size(unsigned long spanned_pages,
                                              unsigned long present_pages)
{
    unsigned long pages = spanned_pages; → Case 1: spanned_pages

    /*
     * Provide a more accurate estimation if there are holes within
     * the zone and SPARSEMEM is in use. If there are holes within the
     * zone, each populated memory region may cost us one or two extra
     * memmap pages due to alignment because memmap pages for each
     * populated regions may not be naturally aligned on page boundary.
     * So the (present_pages >> 4) heuristic is a tradeoff for that.
     */
    if (spanned_pages > present_pages + (present_pages >> 4) &&
        IS_ENABLED(CONFIG_SPARSEMEM))
        pages = present_pages;

    return PAGE_ALIGN(pages * sizeof(struct page)) >> PAGE_SHIFT;
}
```

Case 2: [sparse case] use present_pages

```
Thread 1 hit Breakpoint 2, calc_memmap_size present_pages=786302, spanned_pages
=1048575) at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_al
loc.c:6944
6944 memmap pages = calc_memmap_size(size, freesize);
```

Total present_pages: breakdown

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

```
Early memory node ranges
node 0: [mem 0x0000000000001000-0x000000000009efff]
node 0: [mem 0x0000000000100000-0x00000000bffdffff]
node 0: [mem 0x0000000100000000-0x000000023fffffff] ←
node 1: [mem 0x0000000240000000-0x000000043fffffff]
Zeroed struct page in unavailable ranges: 130 pages
Initmem setup node 0 [mem 0x0000000000001000-0x000000023fffffff]
On node 0 totalpages: 2097022
DMA32 zone: 12286 pages used for memmap
DMA32 zone: 21 pages reserved
DMA32 zone: 786302 pages, LIFO batch:63
Normal zone: 20480 pages used for memmap ←
Normal zone: 1310720 pages, LIFO batch:63
Initmem setup node 1 [mem 0x0000000240000000-0x000000043fffffff]
On node 1 totalpages: 2097152
Normal zone: 32768 pages used for memmap
Normal zone: 2097152 pages, LIFO batch:63
```

spanned_pages case

```
static unsigned long __init calc_memmap_size(unsigned long spanned_pages,
                                             unsigned long present_pages)
{
    unsigned long pages = spanned_pages; → Case 1: spanned_pages

    /*
     * Provide a more accurate estimation if there are holes within
     * the zone and SPARSEMEM is in use. If there are holes within the
     * zone, each populated memory region may cost us one or two extra
     * memmap pages due to alignment because memmap pages for each
     * populated regions may not be naturally aligned on page boundary.
     * So the (present_pages >> 4) heuristic is a tradeoff for that.
     */
    if (spanned_pages > present_pages + (present_pages >> 4) &&
        IS_ENABLED(CONFIG_SPARSEMEM))
        pages = present_pages; ← Case 2: [sparse case] use present_pages

    return PAGE_ALIGN(pages * sizeof(struct page)) >> PAGE_SHIFT;
}
```

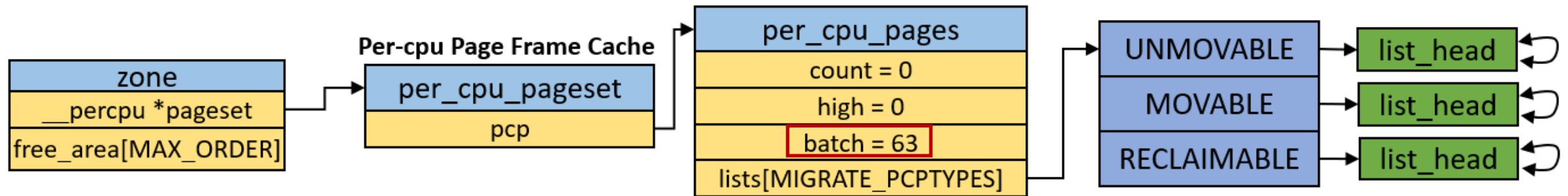
```
Thread 1 hit Breakpoint 2, calc_memmap_size (present_pages=1310720, spanned_page
s=1310720) at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_a
lloc.c:6944
6944      memmap_pages = calc_memmap_size(size, freesize);
```

Total present_pages: breakdown

- * Based on kernel 5.11 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * earlyprintk=serial,ttyS0 console=ttyS0
loglevel=8 nokaslr

```
Early memory node ranges
node 0: [mem 0x00000000000001000-0x00000000000009efff]
node 0: [mem 0x00000000000010000-0x000000000bffdffff]
node 0: [mem 0x0000000100000000-0x0000000023fffffff]
node 1: [mem 0x0000000240000000-0x0000000043fffffff]
Zeroed struct page in unavailable ranges: 130 pages
Initmem setup node 0 [mem 0x00000000000001000-0x0000000023fffffff]
On node 0 totalpages: 2097022
  DMA32 zone: 12286 pages used for memmap
  DMA32 zone: 21 pages reserved
  DMA32 zone: 786302 pages, LIFO batch:63
  Normal zone: 20480 pages used for memmap
  Normal zone: 1310720 pages, LIFO batch:63
Initmem setup node 1 [mem 0x0000000240000000-0x0000000043fffffff]
On node 1 totalpages: 2097152
  Normal zone: 32768 pages used for memmap
  Normal zone: 2097152 pages, LIFO batch:63
```

pcp: per-cpu-pages pool



batch: pre-allocate 'batch' pages for per_cpu_pages if per_cpu_pages is empty

Page frame allocator: watermark

1. `_watermark[]` configuration
2. `lowmem_reserve[MAX_NR_ZONES]`
3. `__zone_watermark_ok()`
 - `free_pages` and `_watermark[]` – adjustment

Zone - watermarks

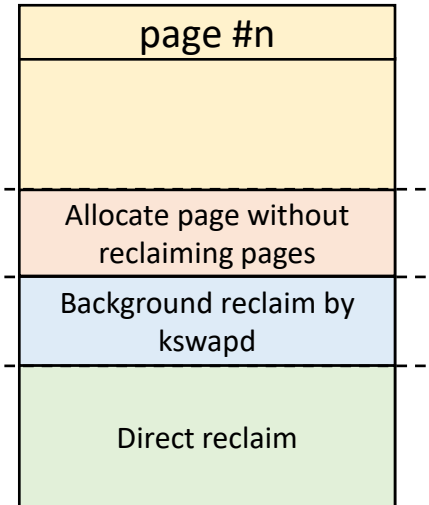
```
init_per_zone_wmark_min [mm/page_alloc.c]
|- nr_free_buffer_pages
|- nr_free_buffer_pages
|- nr_free_zone_pages
  /* 1. Get number of pages beyond
   *   high watermark.
   * 2. Update 'min_free_kbytes'
   */
|- setup_per_zone_wmarks
  |- __setup_per_zone_wmarks
    Calculate watermarks for all zones
    |- calculate_totalreserve_pages
|- refresh_zone_stat_thresholds
|- setup_per_zone_lowmem_reserve
  Calculate each 'zone->lowmem_reserve[]'
  |- calculate_totalreserve_pages
|- khugepaged_min_free_kbytes_update
```

$_watermark[WMARK_HIGH] = _watermark[WMARK_MIN] * (3/2)$ ^{Note 1}

$_watermark[WMARK_LOW] = _watermark[WMARK_MIN] * (5/4)$ ^{Note 1}

$_watermark[WMARK_MIN] = \text{min_free_pages} * \frac{\text{Zone's managed pages}}{\text{All Zones' managed pages}}$

Zone (!highmem_zone)



Note 1: This is the old formula. The new formula includes kswapd watermarks distance according to the scale factor.

Zone – Update _watermark[]

```
init_per_zone_wmark_min [mm/page_alloc.c]
|- nr_free_buffer_pages
  |- nr_free_buffer_pages
    |- nr_free_zone_pages
      /* 1. Get number of pages beyond
       *   high watermark.
       * 2. Update 'min_free_kbytes'
       */
    |- setup_per_zone_wmarks
      |- __setup_per_zone_wmarks
        Calculate watermarks for all zones
      |- calculate_totalreserve_pages
    |- refresh_zone_stat_thresholds
    |- setup_per_zone_lowmem_reserve
      Calculate each 'zone->lowmem_reserve[]'
    |- calculate_totalreserve_pages
    |- khugepaged_min_free_kbytes_update
```

Stage 1: update 'min_free_kbytes' and _watermark[]

Stage 2 (huge page): update 'min_free_kbytes' and _watermark[]

Let's check nr_free_buffer_pages()

[Stage 1] Zone – watermark: min_free_kbytes:

[min_free_kbytes]

- Force the page frame allocator to keep a minimum number of kilobytes free.
- The page frame allocator uses this number to compute a watermark[WMARK_MIN] value.

```
int __meminit init_per_zone_wmark_min(void)
{
    unsigned long lowmem_kbytes;
    int new_min_free_kbytes;

    lowmem_kbytes = nr_free_buffer_pages() * (PAGE_SIZE >> 10);
    new_min_free_kbytes = int_sqrt(lowmem_kbytes * 16);

    if (new_min_free_kbytes > user_min_free_kbytes) {
        min_free_kbytes = new_min_free_kbytes;
        if (min_free_kbytes < 128)
            min_free_kbytes = 128;
        if (min_free_kbytes > 262144)
            min_free_kbytes = 262144;
    } else {
        pr_warn("min_free_kbytes is not updated to %d because user defin
ed value %d is preferred\n",
                new_min_free_kbytes, user_min_free_kbytes);
    }
    setup_per_zone_wmarks();
    refresh_zone_stat_thresholds();
    setup_per_zone_lowmem_reserve();
+--- 9 lines: #ifdef CONFIG_NUMA-----
}
mm/page_alloc.c                                8003,1                89%
```

- $\text{nr_free_buffer_pages}() = (764421 - 0) + (3335722 - 0) = 4100143$ pages: Get number of pages beyond high watermark
- $\text{lowmem_kbytes} = 4100143 * 4 = 16400572$
- $\text{new_min_free_kbytes} = \text{min_free_kbytes} = \text{int_sqrt}(16400572 * 16) = \text{floor}(\text{sqrt}(16400572 * 16)) = 16199$

```
(gdb) p node_data[0]->node_zones
$5 = {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x22148,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1,
  managed_pages = {
    counter = 764421
  },
  spanned_pages = 1048575,
  present_pages = 786302,
  name = 0xffffffff81915e56 "DMA32",
  ...
}, {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x221a8,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1048576,
  managed_pages = {
    counter = 3335722
  },
  spanned_pages = 3407872,
  present_pages = 3407872,
  name = 0xffffffff81916917 "Normal",
  ...
}, {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x1d1c0 <boot_pageset>,
  pageset_high = 0,
  pageset_batch = 1,
  zone_start_pfn = 0,
  managed_pages = {
    counter = 0
  },
  spanned_pages = 0,
  present_pages = 0,
  name = 0xffffffff819168f0 "Movable",
  ...
}
```

[Stage 1] Zone – watermark: min_free_kbytes:

```
int __meminit init_per_zone_wmark_min(void)
{
    unsigned long lowmem_kbytes;
    int new_min_free_kbytes;

    lowmem_kbytes = nr_free_buffer_pages() * (PAGE_SIZE >> 10);
    new_min_free_kbytes = int_sqrt(lowmem_kbytes * 16);

    if (new_min_free_kbytes > user_min_free_kbytes) {
        min_free_kbytes = new_min_free_kbytes;
        if (min_free_kbytes < 128)
            min_free_kbytes = 128;
        if (min_free_kbytes > 262144)
            min_free_kbytes = 262144;
    } else {
        pr_warn("min_free_kbytes is not updated to %d because user defin
ed value %d is preferred\n",
                new_min_free_kbytes, user_min_free_kbytes);
    }
    setup_per_zone_wmarks();
    refresh_zone_stat_thresholds();
    setup_per_zone_lowmem_reserve();
+-- 9 lines: #ifdef CONFIG_NUMA-----
}
mm/page_alloc.c                               8003,1          89%
```

$\text{nr_free_buffer_pages}() = (764421 - 0) + (3335722 - 0) = 4100143$

$\text{lowmem_kbytes} = 4100143 * 4 = 16400572$

$\text{new_min_free_kbytes} = \text{min_free_kbytes} = \text{int_sqrt}(16400572 * 16) = \text{floor}(\text{sqrt}(16400572 * 16)) = 16199$

```
(gdb) bt 2
#0  setup_per_zone_wmarks ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/include/linux/sp
inlock.h:354
#1  0xffffffff81371017 in init_per_zone_wmark_min ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/page_alloc.c:
8025
(More stack frames follow...)
(gdb) p min_free_kbytes
$6 = 16199
```

```
(gdb) p node_data[0]->node_zones
$5 = {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x22148,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1,
  managed_pages = {
    counter = 764421
  },
  spanned_pages = 1048575,
  present_pages = 786302,
  name = 0xffffffff81915e56 "DMA32",
  ...
}, {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x221a8,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1048576,
  managed_pages = {
    counter = 3335722
  },
  spanned_pages = 3407872,
  present_pages = 3407872,
  name = 0xffffffff81916917 "Normal",
  ...
}, {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x1d1c0 <boot_pageset>,
  pageset_high = 0,
  pageset_batch = 1,
  zone_start_pfn = 0,
  managed_pages = {
    counter = 0
  },
  spanned_pages = 0,
  present_pages = 0,
  name = 0xffffffff819168f0 "Movable",
  ...
}
```

[Stage 1] Zone – Update _watermark[]

```
init_per_zone_wmark_min [mm/page_alloc.c]
|- nr_free_buffer_pages
  |- nr_free_buffer_pages
    |- nr_free_zone_pages
      /* 1. Get number of pages beyond
       *   high watermark.
       * 2. Update 'min_free_kbytes'
       */
- setup_per_zone_wmarks
  |- __setup_per_zone_wmarks
    Calculate watermarks for all zones
  |- calculate_totalreserve_pages
|- refresh_zone_stat_thresholds
|- setup_per_zone_lowmem_reserve
  Calculate each 'zone->lowmem_reserve[]'
  |- calculate_totalreserve_pages
|- khugepaged_min_free_kbytes_update
```

$_watermark[WMARK_HIGH] = _watermark[WMARK_MIN] * (3/2)$ Note 1

$_watermark[WMARK_LOW] = _watermark[WMARK_MIN] * (5/4)$ Note 1

$_watermark[WMARK_MIN] = \text{min_free_pages} * \frac{\text{Zone's managed pages}}{\text{All Zones' managed pages}}$

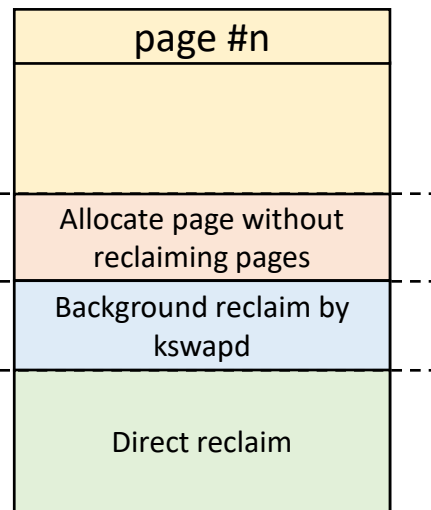
-- Normal Zone -- $_watermark[WMARK_MIN]$

$$u64\left(\frac{16199}{4}\right) * \frac{3335722}{(764421 + 3335722)} = 4049 * \frac{3335722}{4100143} = 3294$$

```
(gdb) p min_free_kbytes
$3 = 16199
```

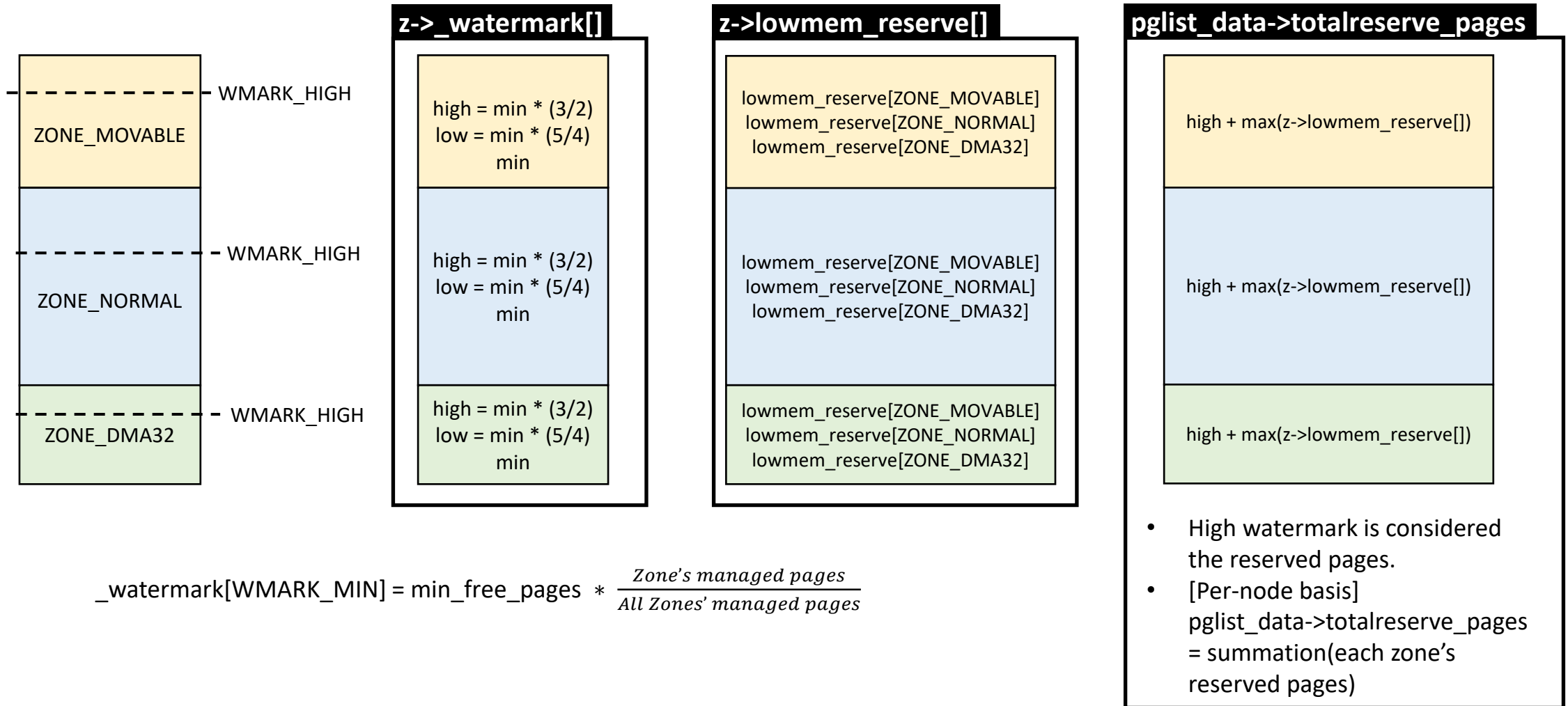
Note 1: This is the old formula. The new formula includes kswapd watermarks distance according to the scale factor.

Zone (!highmem_zone)



```
(gdb) p node_data[0]->node_zones
$2 = {{
  _watermark = {754, 1518, 2282},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x22148,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1,
  managed_pages = {
    counter = 764421
  },
  spanned_pages = 1048575,
  present_pages = 786302,
  name = 0xffffffff81915e56 "DMA32",
  ...
}, {
  _watermark = {3294, 6629, 9964},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x221a8,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1048576,
  managed_pages = {
    counter = 3335722
  },
  spanned_pages = 3407872,
  present_pages = 3407872,
  name = 0xffffffff81916917 "Normal",
  ...
}, {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x1d1c0 <boot_pageset>,
  pageset_high = 0,
  pageset_batch = 1,
  zone_start_pfn = 0,
  managed_pages = {
    counter = 0
  },
  spanned_pages = 0,
  present_pages = 0,
  name = 0xffffffff819168f0 "Movable",
  ...
}}}
```

[Stage 1] calculate_totalreserve_pages()



Update pglist_data->totalreserve_pages by iterating each zone

[Stage 1] calculate_totalreserve_pages()

z->lowmem_reserve[]

lowmem_reserve[ZONE_MOVABLE]
lowmem_reserve[ZONE_NORMAL]
lowmem_reserve[ZONE_DMA32]

lowmem_reserve[ZONE_MOVABLE]
lowmem_reserve[ZONE_NORMAL]
lowmem_reserve[ZONE_DMA32]

lowmem_reserve[ZONE_MOVABLE]
lowmem_reserve[ZONE_NORMAL]
lowmem_reserve[ZONE_DMA32]

pglist_data->totalreserve_pages

high + max(z->lowmem_reserve[])

high + max(z->lowmem_reserve[])

high + max(z->lowmem_reserve[])

- High watermark is considered the reserved pages.
- [Per-node basis]
pglist_data->totalreserve_pages = summation(each zone's reserved pages)

pglist_data->totalreserve_pages = (2282 + max(0, 0, 0)) + (9964 + max(0,0,0)) = 12246

(gdb) p node_data[0]->totalreserve_pages
\$6 = 12246

```
(gdb) p node_data[0]->node_zones
$2 = {
  _watermark = {754, 1518, 2282},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x22148,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1,
  managed_pages = {
    counter = 764421
  },
  spanned_pages = 1048575,
  present_pages = 786302,
  name = 0xffffffff81915e56 "DMA32",
  ...
}, {
  _watermark = {3294, 6629, 9964},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x221a8,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1048576,
  managed_pages = {
    counter = 3335722
  },
  spanned_pages = 3407872,
  present_pages = 3407872,
  name = 0xffffffff81916917 "Normal",
  ...
}, {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x1d1c0 <boot_pageset>,
  pageset_high = 0,
  pageset_batch = 1,
  zone_start_pfn = 0,
  managed_pages = {
    counter = 0
  },
  spanned_pages = 0,
  present_pages = 0,
  name = 0xffffffff819168f0 "Movable",
  ...
} }
```

[Stage 1] lowmem_reserve[MAX_NR_ZONES]

```
init_per_zone_wmark_min [mm/page_alloc.c]
|- nr_free_buffer_pages
  |- nr_free_buffer_pages
  |- nr_free_zone_pages
    /* 1. Get number of pages beyond
     *    high watermark.
     * 2. Update 'min_free_kbytes'
     */
|- setup_per_zone_wmarks
  |- __setup_per_zone_wmarks
    Calculate watermarks for all zones
  |- calculate_totalreserve_pages
|- refresh_zone_stat_thresholds
|- setup_per_zone_lowmem_reserve
  Calculate each 'zone->lowmem_reserve[]'
  |- calculate_totalreserve_pages
|- khugepaged_min_free_kbytes_update
```

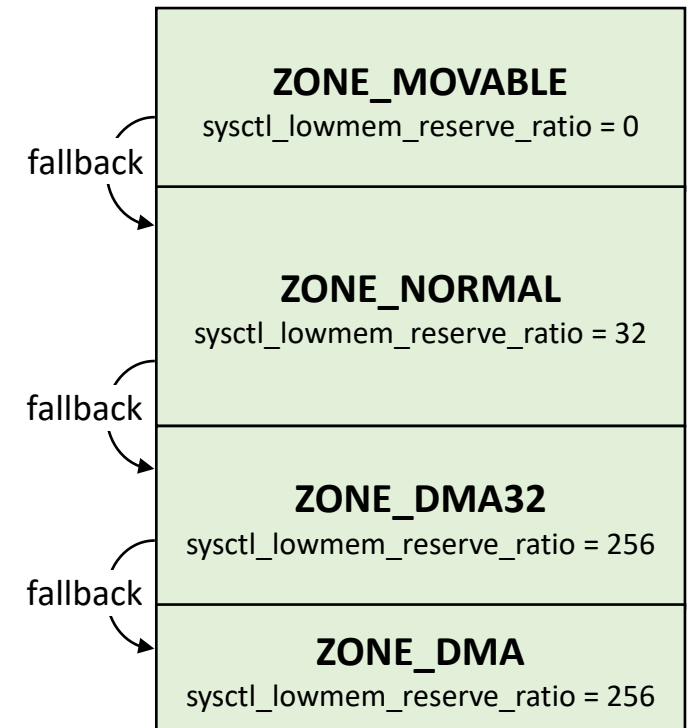
Issue Statement

- [Fallback Mechanism] Insufficient memory of higher zone → Allocate memory from lower zone
 - The memory of the lower zone might be exhausted because of the request of the higher zone.
 - Example (gfp flag)
 - GFP_DMA: ZONE_DMA
 - GFP_DMA32: ZONE_DMA32 -> ZONE_DMA
 - Otherwise: ZONE_NORMAL -> ZONE_DMA32 -> ZONE_DMA
 - Scenario
 - The memory allocation from higher zone could be the pinned pages via mlock().

Solution

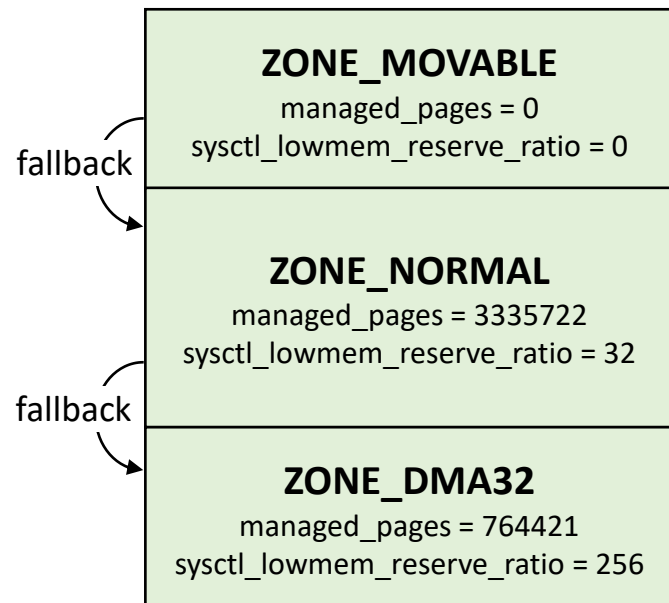
- lowmem_reserve[MAX_NR_ZONES]
 - Ensure a certain amount of free pages of the lower zone are reserved.

zone
_watermark[NR_WMARK]
lowmem_reserve[MAX_NR_ZONES]
node
zone_pgdat
zone_start_pfn
managed_pages
spanned_pages
present_pages
free_area[MAX_ORDER]



[Stage 1] lowmem_reserve[MAX_NR_ZONES]

```
init_per_zone_wmark_min [mm/page_alloc.c]
|- nr_free_buffer_pages
  |- nr_free_buffer_pages
    |- nr_free_zone_pages
      /* 1. Get number of pages beyond
       *   high watermark.
       * 2. Update 'min_free_kbytes'
       */
|- setup_per_zone_wmarks
  |- __setup_per_zone_wmarks
    Calculate watermarks for all zones
  ① |- calculate_totalreserve_pages
    |- refresh_zone_stat_thresholds
      ② |- setup_per_zone_lowmem_reserve
        Calculate each 'zone->lowmem_reserve[]'
      ② |- calculate_totalreserve_pages
    |- khugepaged_min_free_kbytes_update
```



	lowmem_reserve[ZONE_DMA32]	lowmem_reserve[ZONE_NORMAL]	lowmem_reserve[ZONE_MOVABLE]
ZONE_DMA32	$\frac{0}{256} = 0$	$\frac{3335722}{256} = 13030$	$\frac{(3335722 + 0)}{256} = 13030$
ZONE_NORMAL	$\frac{0}{32} = 0$	$\frac{0}{32} = 0$	$\frac{0}{32} = 0$
ZONE_MOVABLE	0	0	0

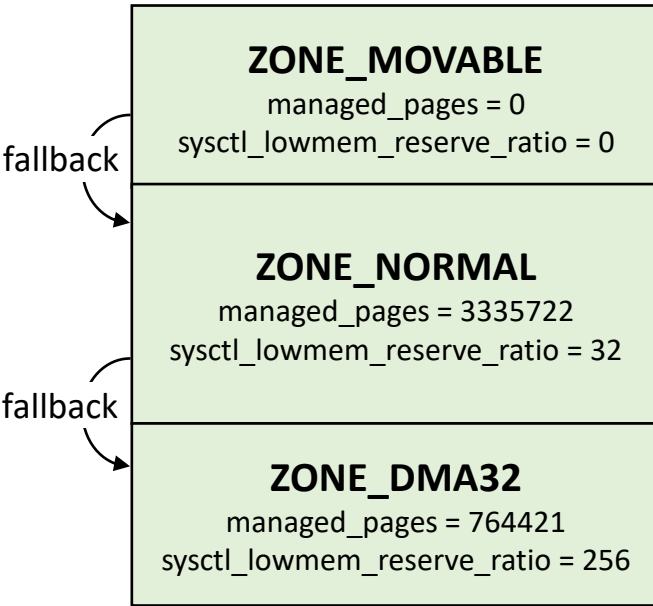
$\text{pglist_data->totalreserve_pages} = (2282 + \max(0, 13030, 13030)) + (9964 + \max(0, 0, 0)) = 25276$

① (gdb) p node_data[0]->totalreserve_pages
\$6 = 12246

② (gdb) p node_data[0]->totalreserve_pages
\$9 = 25276

```
(gdb) p node_data[0]->node_zones
$8 = {{
  _watermark = {754, 1518, 2282},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 13030, 13030},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x22148,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1,
  managed_pages = {
    counter = 764421
  },
  spanned_pages = 1048575,
  present_pages = 786302,
  name = 0xffffffff81915e56 "DMA32",
  ...
}, {
  _watermark = {3294, 6629, 9964},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x221a8,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1048576,
  managed_pages = {
    counter = 3335722
  },
  spanned_pages = 3407872,
  present_pages = 3407872,
  name = 0xffffffff81916917 "Normal",
  ...
}, {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x1d1c0 <boot_pageset>,
  pageset_high = 0,
  pageset_batch = 1,
  zone_start_pfn = 0,
  managed_pages = {
    counter = 0
  },
  spanned_pages = 0,
  present_pages = 0,
  name = 0xffffffff819168f0 "Movable",
  ...
}}}
```

Zone – lowmem_reserve[MAX_NR_ZONES]: Usage



```
static inline bool zone_watermark_fast(struct zone *z, unsigned int order,
                                       unsigned long mark, int highest_zoneidx,
                                       unsigned int alloc_flags, gfp_t gfp_mask)
{
    long free_pages;

    free_pages = zone_page_state(z, NR_FREE_PAGES);

    if (!order) {
        long fast_free;

        fast_free = free_pages;
        fast_free -= __zone_watermark_unusable_free(z, 0, alloc_flags);
        if (fast_free > mark + z->lowmem_reserve[highest_zoneidx])
            return true;
    }

    ...

mm/page_alloc.c 3701,1 41%
```

```
bool __zone_watermark_ok(struct zone *z, unsigned int order, unsigned long mark,
                        int highest_zoneidx, unsigned int alloc_flags,
                        long free_pages)
{
    long min = mark;

    ...
    /* free_pages may go negative - that's OK */
    free_pages -= __zone_watermark_unusable_free(z, order, alloc_flags);

    ...
    if (free_pages <= min + z->lowmem_reserve[highest_zoneidx])
        return false;

    ...

mm/page_alloc.c 3633,1 40%
```

```
(gdb) p node_data[0]->node_zones
$8 = {{
    _watermark = {754, 1518, 2282},
    watermark_boost = 0,
    nr_reserved_highatomic = 0,
    lowmem_reserve = {0, 13030, 13030},
    node = 0,
    zone_pgdat = 0xffff88843ffdf000,
    pageset = 0x22148,
    pageset_high = 378,
    pageset_batch = 63,
    zone_start_pfn = 1,
    managed_pages = {
        counter = 764421
    },
    spanned_pages = 1048575,
    present_pages = 786302,
    name = 0xffffffff81915e56 "DMA32",
    ...
}, {
    _watermark = {3294, 6629, 9964},
    watermark_boost = 0,
    nr_reserved_highatomic = 0,
    lowmem_reserve = {0, 0, 0},
    node = 0,
    zone_pgdat = 0xffff88843ffdf000,
    pageset = 0x221a8,
    pageset_high = 378,
    pageset_batch = 63,
    zone_start_pfn = 1048576,
    managed_pages = {
        counter = 3335722
    },
    spanned_pages = 3407872,
    present_pages = 3407872,
    name = 0xffffffff81916917 "Normal",
    ...
}, {
    _watermark = {0, 0, 0},
    watermark_boost = 0,
    nr_reserved_highatomic = 0,
    lowmem_reserve = {0, 0, 0},
    node = 0,
    zone_pgdat = 0xffff88843ffdf000,
    pageset = 0x1d1c0 <boot_pageset>,
    pageset_high = 0,
    pageset_batch = 1,
    zone_start_pfn = 0,
    managed_pages = {
        counter = 0
    },
    spanned_pages = 0,
    present_pages = 0,
    name = 0xffffffff819168f0 "Movable",
    ...
}}}
```

[Stage 2] lowmem_reserve[MAX_NR_ZONES] (CONFIG_TRANSPARENT_HUGEPAGE=y)

```
init_per_zone_wmark_min [mm/page_alloc.c]
|- nr_free_buffer_pages
|- nr_free_buffer_pages
|- nr_free_zone_pages
/* 1. Get number of pages beyond
 * high watermark.
 * 2. Update 'min_free_kbytes'
 */
|- setup_per_zone_wmarks
|- __setup_per_zone_wmarks
Calculate watermarks for all zones
|- calculate_totalreserve_pages
|- refresh_zone_stat_thresholds
|- setup_per_zone_lowmem_reserve
Calculate each 'zone->lowmem_reserve[]'
1|- calculate_totalreserve_pages
2|- khugepaged_min_free_kbytes_update
|- set_recommended_min_free_kbytes
```

```
static void set_recommended_min_free_kbytes(void)
{
+--- 15 lines: struct zone *zone;-----
/* Ensure 2 pageblocks are free to assist fragmentation avoidance */
recommended_min = pageblock_nr_pages * nr_zones * 2;

/*
 * Make sure that on average at least two pageblocks are almost free
 * of another type, one for a migratetype to fall back to and a
 * second to avoid subsequent fallbacks of other types There are 3
 * MIGRATE_TYPES we care about.
 */
recommended_min += pageblock_nr_pages * nr_zones *
MIGRATE_PCPTYPES * MIGRATE_PCPTYPES;

/* don't ever allow to reserve more than 5% of the lowmem */
recommended_min = min(recommended_min,
(unsigned long) nr_free_buffer_pages() / 20);
recommended_min <= (PAGE_SHIFT-10);

if (recommended_min > min_free_kbytes) {
if (user_min_free_kbytes >= 0)
pr_info("raising min_free_kbytes from %d to %lu to help
transparent hugepage allocations\n",
min_free_kbytes, recommended_min);

min_free_kbytes = recommended_min;
}
}
} setup_per_zone_wmarks();
}
mm/khugepaged.c 2317,1-8 98%
```

nr_free_buffer_pages(): Get number of pages beyond high watermark

(gdb) p min_free_kbytes
\$3 = 16199

(gdb) p min_free_kbytes
\$16 = 45056

$pglist_data \rightarrow totalreserve_pages = (3628 + \max(0, 13030, 13030)) + (15833 + \max(0, 0, 0)) = 32491$

1 (gdb) p node_data[0]->totalreserve_pages
\$9 = 25276

2 (gdb) p node_data[0]->totalreserve_pages
\$14 = 32491

```
(gdb) p node_data[0]->node_zones
$1 = {{
  _watermark = {2100, 2864, 3628},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 13030, 13030},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x22148,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1,
  managed_pages = {
    counter = 764421
  },
  spanned_pages = 1048575,
  present_pages = 786302,
  name = 0xffffffff81915e56 "DMA32",
  ...
}, {
  _watermark = {9163, 12498, 15833},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x221a8,
  pageset_high = 378,
  pageset_batch = 63,
  zone_start_pfn = 1048576,
  managed_pages = {
    counter = 3335722
  },
  spanned_pages = 3407872,
  present_pages = 3407872,
  name = 0xffffffff81916917 "Normal",
  ...
}, {
  _watermark = {0, 0, 0},
  watermark_boost = 0,
  nr_reserved_highatomic = 0,
  lowmem_reserve = {0, 0, 0},
  node = 0,
  zone_pgdat = 0xffff88843ffdf000,
  pageset = 0x1d1c0 <boot_pageset>,
  pageset_high = 0,
  pageset_batch = 1,
  zone_start_pfn = 0,
  managed_pages = {
    counter = 0
  },
  spanned_pages = 0,
  present_pages = 0,
  name = 0xffffffff819168f0 "Movable",
  ...
}}
[] }
```

[Stage 2] lowmem_reserve[MAX_NR_ZONES] (CONFIG_TRANSPARENT_HUGEPAGE=y)

```
init_per_zone_wmark_min [mm/page_alloc.c]
|- nr_free_buffer_pages
  |- nr_free_buffer_pages
    |- nr_free_zone_pages
      /* 1. Get number of pages beyond
       *   high watermark.
       * 2. Update 'min_free_kbytes'
       */
|- setup_per_zone_wmarks
  |- __setup_per_zone_wmarks
    Calculate watermarks for all zones
    |- calculate_totalreserve_pages
|- refresh_zone_stat_thresholds
|- setup_per_zone_lowmem_reserve
  Calculate each 'zone->lowmem_reserve[]'
  |- calculate_totalreserve_pages
  |- khugepaged_min_free_kbytes_update
  |- set_recommended_min_free_kbytes
```

```
void khugepaged_min_free_kbytes_update(void)
{
    mutex_lock(&khugepaged_mutex);
    if (khugepaged_enabled() && khugepaged_thread)
        set_recommended_min_free_kbytes();
    mutex_unlock(&khugepaged_mutex);
}
mm/khugepaged.c
```

[gdb] tip: watchpoint

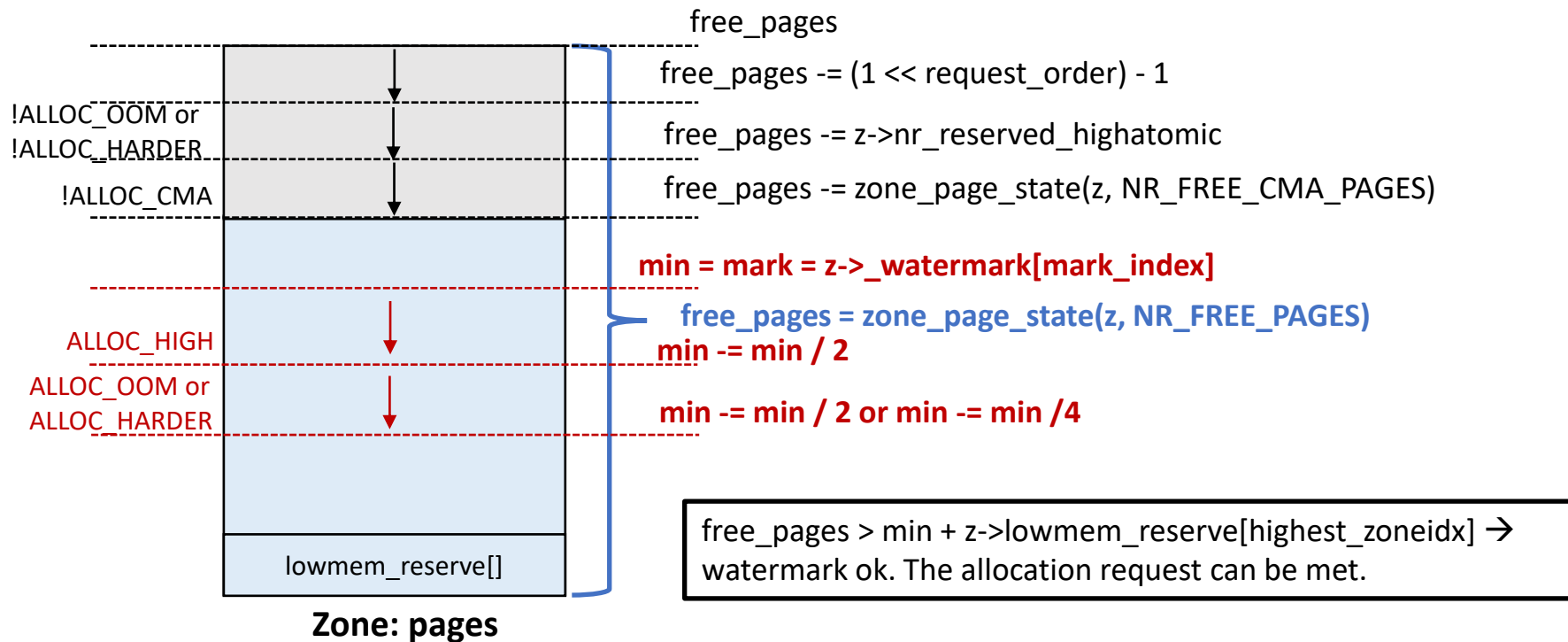
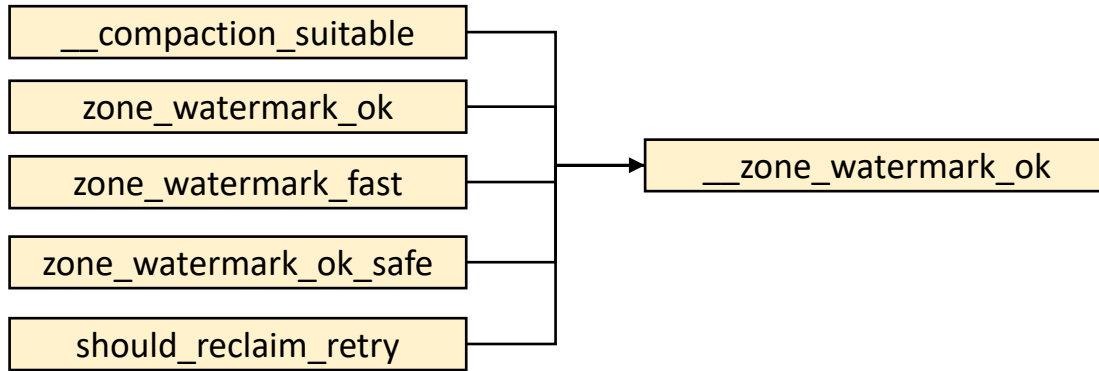
```
Thread 1 hit Hardware watchpoint 3: *(&min_free_kbytes)

Old value = <unreadable>
New value = 45056
set_recommended_min_free_kbytes ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/khugepaged.c:
2317
2317         setup_per_zone_wmarks();
(gdb) bt 3
#0  set_recommended_min_free_kbytes ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/khugepaged.c:
2317
#1  0xffffffff8111cd24 in start_stop_khugepaged ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/khugepaged.c:
2339
#2  0xffffffff81b130b1 in hugepage_init ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/huge_memory.c:
:428
(More stack frames follow...)
(gdb) p min_free_kbytes
$1 = 45056
```

```
/ # cat /proc/sys/vm/min_free_kbytes
45056
```

**set_recommended_min_free_kbytes() is invoked during hugepage_init()
instead of init_per_zone_wmark_min()**

__zone_watermark_ok



Page Frame Allocator

1. Page Allocation
2. fallback zonelist
3. Page De-allocation

Page Frame Allocator: Page Allocation

Preparation

prepare_alloc_pages

Init the members of alloc_context struct:
* highest_zoneidx, zonelist, migratetype, preferred_zoneref and so on.

Fast path

for_next_zone_zonelist_nodemask(): Iterate each zone from zonelist

zone_watermark_fast?

N

node_reclaim

reclaim page caches for a node

Y: free_pages > wmark_low

rmqueue

page order = 0

rmqueue_pcplist

per-cpu page frame cache

else

__rmqueue_smallest

__rmqueue

get_page_from_freelist

__alloc_pages_nodemask

alloc_flags = ALLOC_WMARK_LOW

Slow path

__alloc_pages_slowpath

alloc_flags = gfp_to_alloc_flags(gfp_mask)

Assign flag 'ALLOC_WMARK_MIN' to alloc_flags

wake_all_kswapds

Wake up kswapds if ALLOC_KSWAPD is set

get_page_from_freelist

Get pages if "wmark_min < free_pages < wmark_low"

wake_all_kswapds

Wake up kswapds if ALLOC_KSWAPD is set

get_page_from_freelist

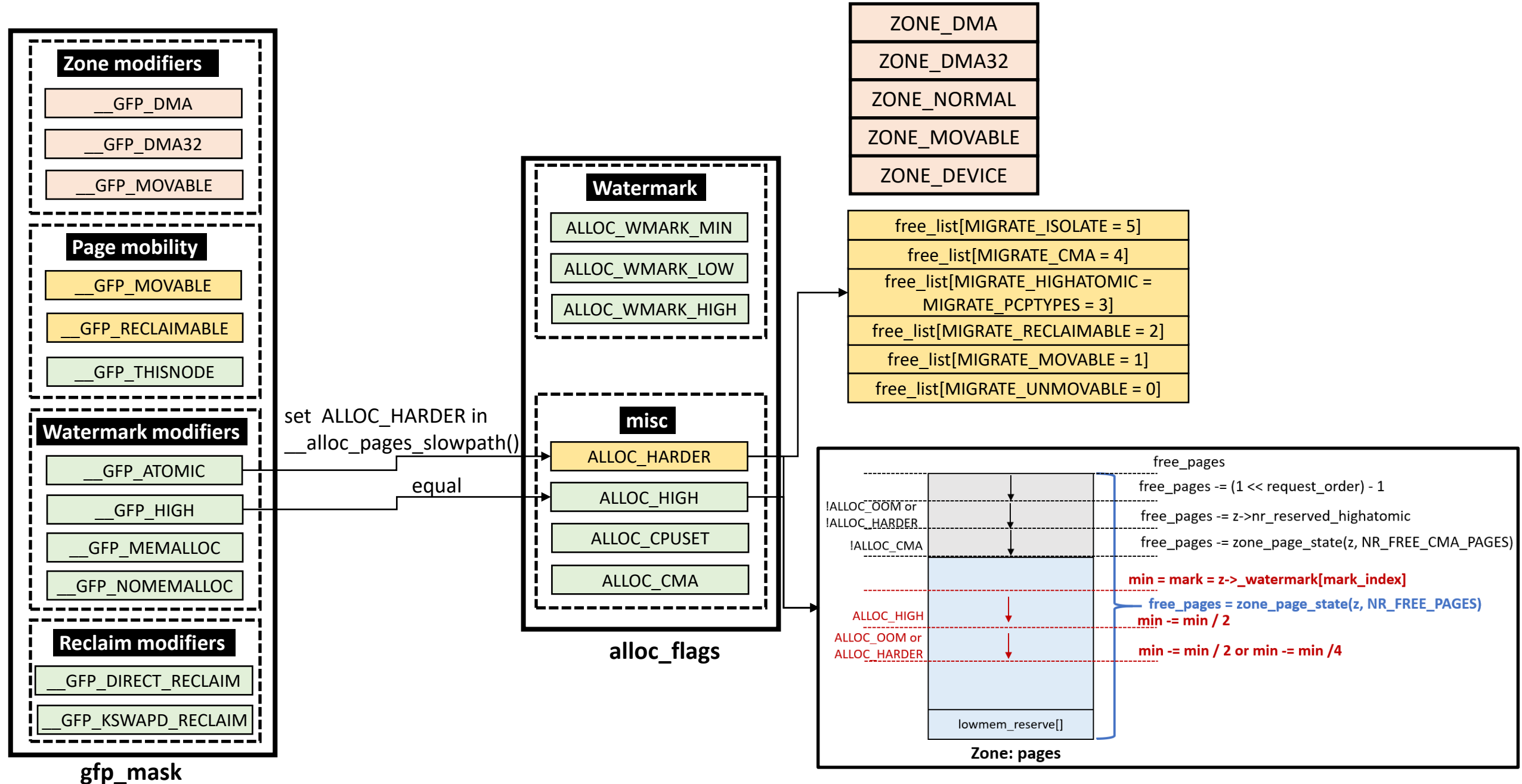
Another try after adjusting zonelist, alloc_flags and nodemask = NULL

__alloc_pages_direct_reclaim

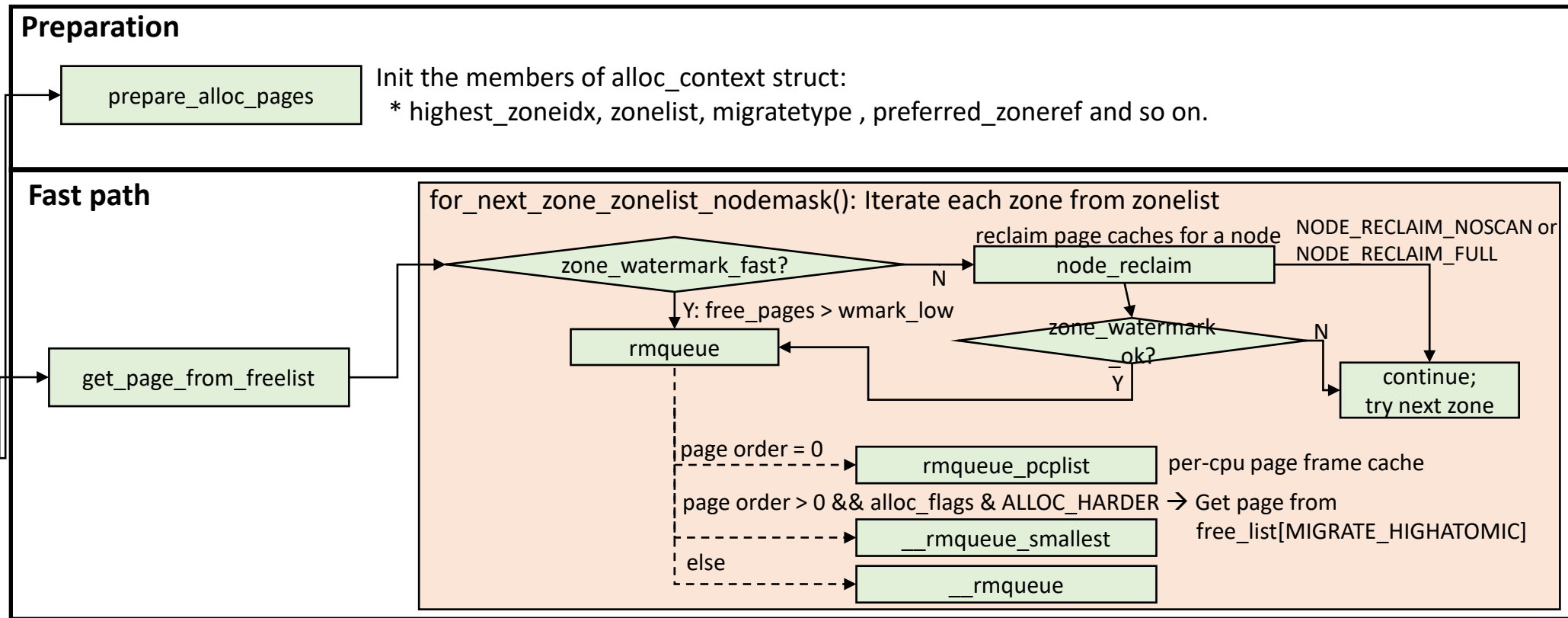
__alloc_pages_direct_compact

__alloc_pages_may_oom

gfp_mask & alloc_flags



Page Frame Allocator: Page Allocation



node_reclaim_mode (checked in node_reclaim function)

- Reclaim memory when a zone runs out of memory
- Representation
 - 0 (default value): disabled
 - 1: Reclaim write dirty pages
 - 2: Reclaim page caches

Page Frame Allocator: Page Allocation - *rmqueue

__rmqueue

(alloc_flags & ALLOC_CMA) && over half of the zone's free is in the CMA areamemory

-> **__rmqueue_cma_fallback**

else

-> **__rmqueue_smallest**

__rmqueue_smallest is failed && (alloc_flags & ALLOC_CMA)

-> **__rmqueue_cma_fallback**

__rmqueue_smallest and __rmqueue_cma_fallback are failed

-> **__rmqueue_fallback**

→ **find_suitable_fallback**

→ **get_page_from_free_area**

→ **steal_suitable_fallback**

→ **move_freepages_block**

→ **move_freepages**

rmqueue

alloc_flags & ALLOC_HARDER

free_list[MIGRATE_HIGHATOMIC]

Based on alloc_flags (gfp flag)

free_list[MIGRATE_ISOLATE = 5]

free_list[MIGRATE_CMA = 4]

**free_list[MIGRATE_HIGHATOMIC =
MIGRATE_PCPTYPES = 3]**

free_list[MIGRATE_RECLAIMABLE = 2]

free_list[MIGRATE_MOVABLE = 1]

free_list[MIGRATE_UNMOVABLE = 0]

__rmqueue_cma_fallback

free_list[MIGRATE_CMA]

__rmqueue_fallback()

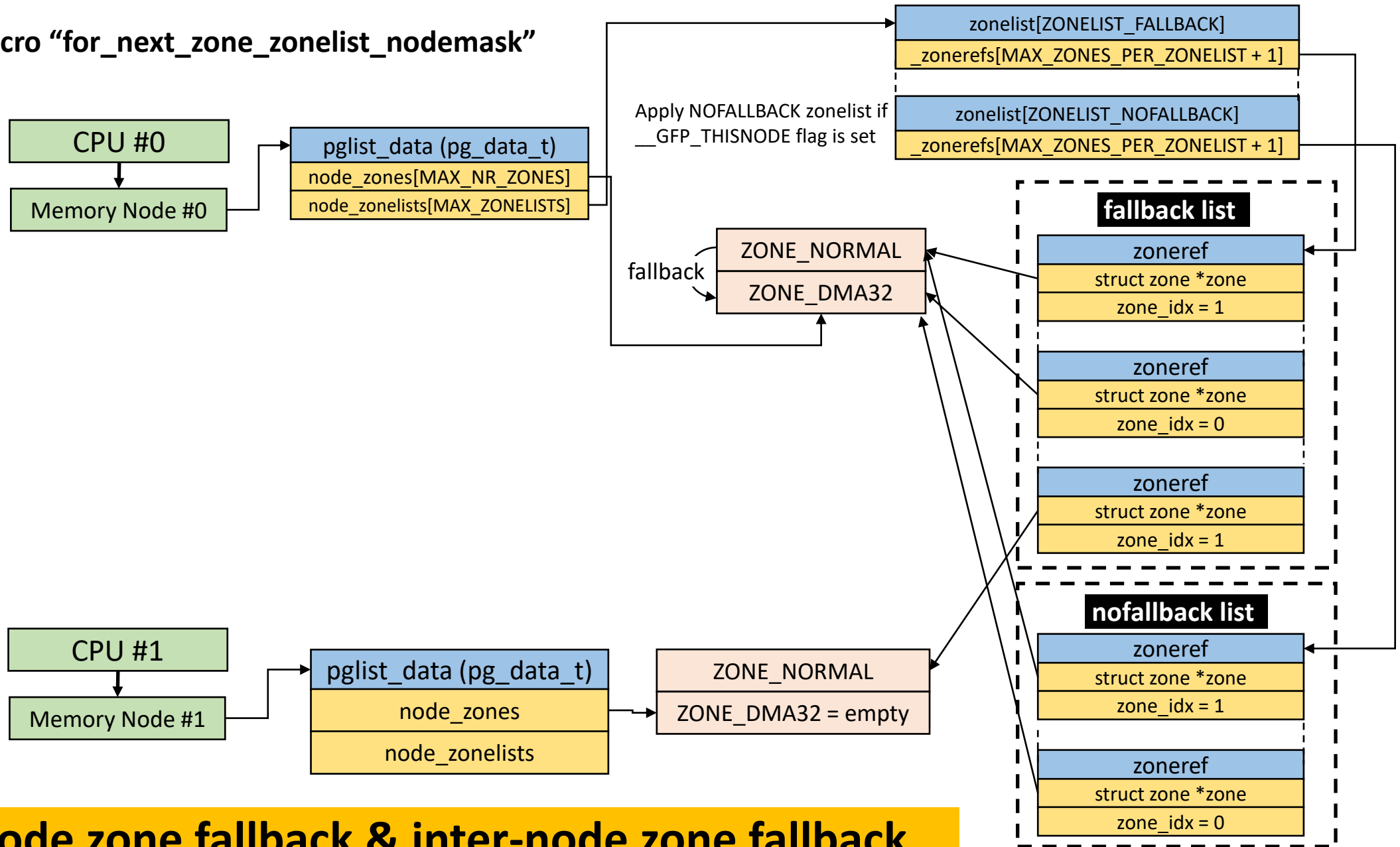
free_list[MIGRATE_RECLAIMABLE]

free_list[MIGRATE_MOVABLE]

free_list[MIGRATE_UNMOVABLE]

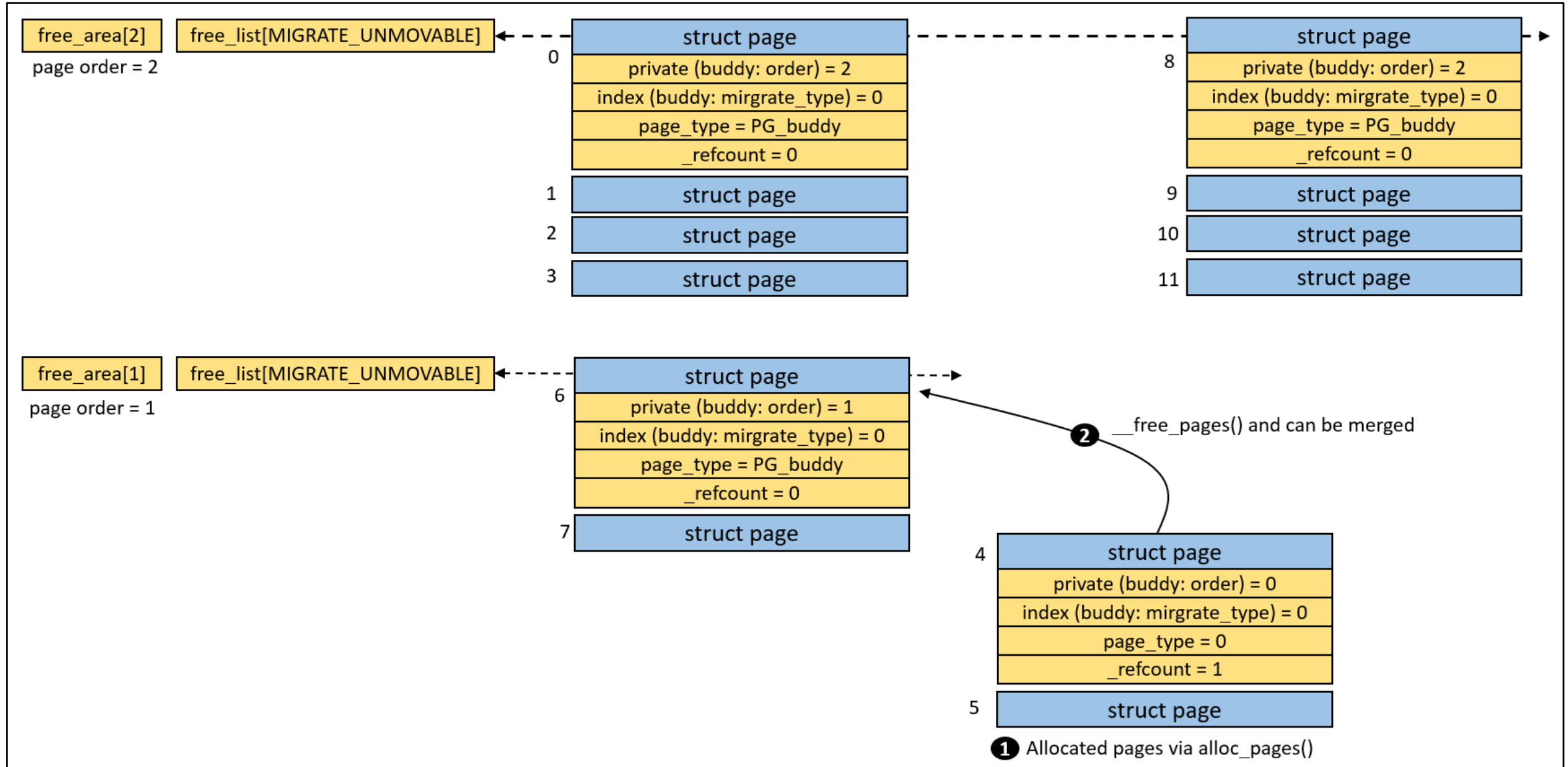
Page Frame Allocator: fallback zone list

* Check macro "for_next_zone_zonelist_nodemask"



Intra-node zone fallback & inter-node zone fallback

Page Frame Allocator: Page Deallocation



Reference

- <https://wdv4758h.github.io/notes/blog/linux-kernel-boot.html>
- <https://www.cnblogs.com/LoyenWang/p/11626237.html>
- <https://pingcap.com/blog/linux-kernel-vs-memory-fragmentation-1>
- <https://www.programmersought.com/article/81176896338/>